

Deep Learning - IV

RRC Summer School 2026

Recap

Convolutional Neural Networks (CNNs)

Images are Numbers



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

What the computer sees

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

An image is just a matrix of numbers $[0,255]$!
i.e., $1080 \times 1080 \times 3$ for an RGB image

Manual Feature Extraction

Domain
knowledge

Define features

Detect features
to classify

Viewpoint variation



Scale variation



Deformation



Occlusion



Illumination conditions



Background clutter



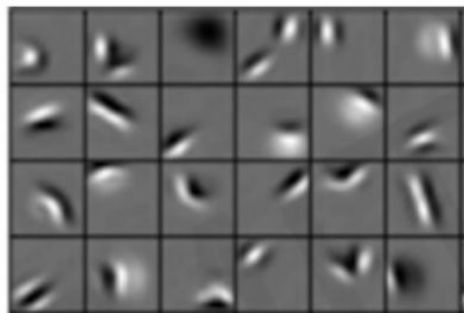
Intra-class variation



Learning Feature Representations

Can we learn a **hierarchy of features** directly from the data instead of hand engineering?

Low level features



Edges, dark spots

Mid level features



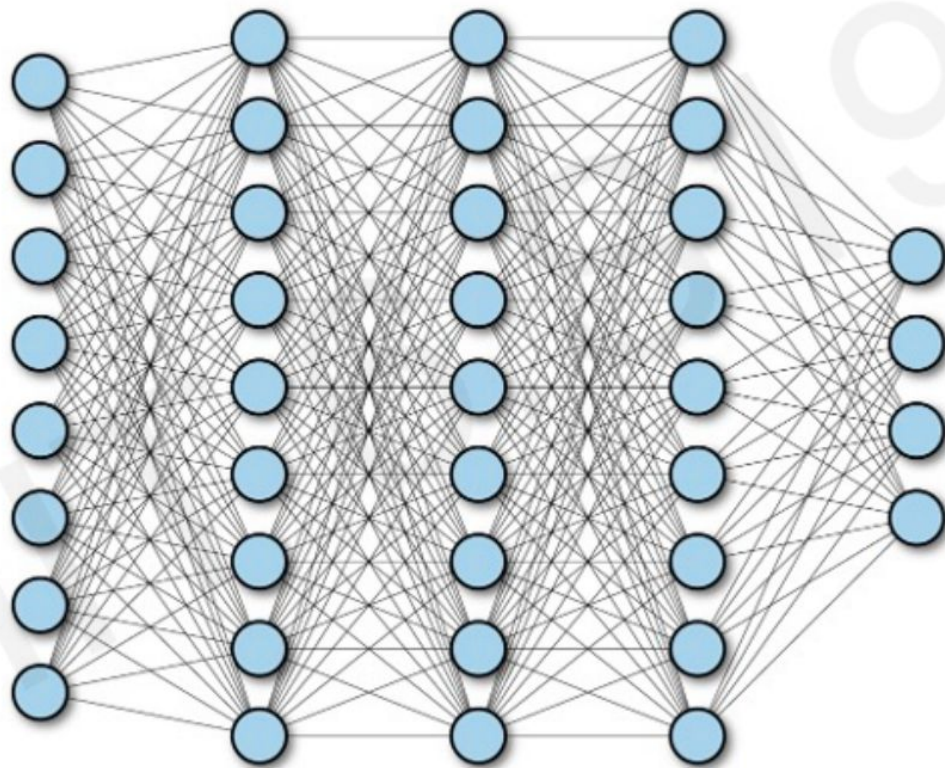
Eyes, ears, nose

High level features



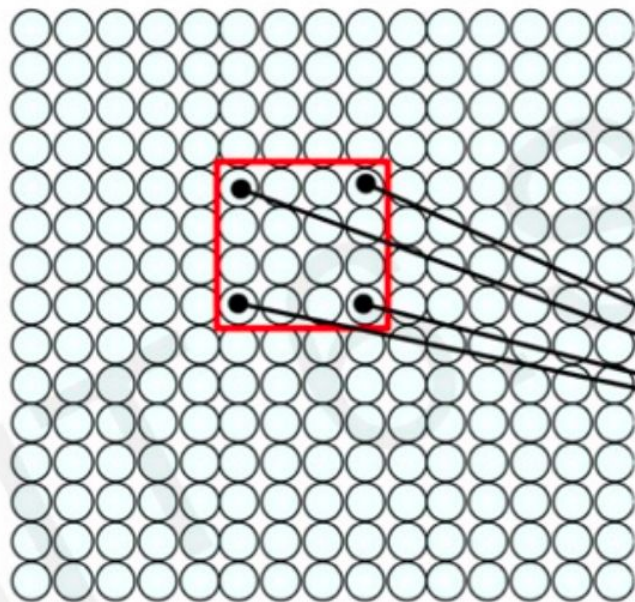
Facial structure

Fully Connected Neural Network



Using Spatial Structure

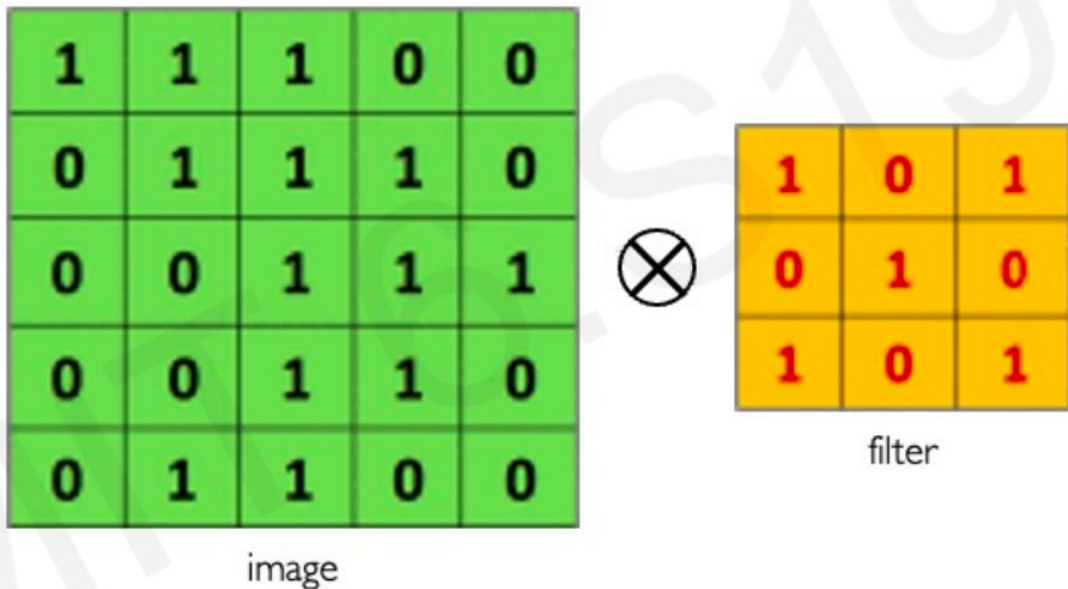
Input: 2D image.
Array of pixel values



Idea: connect patches of input
to neurons in hidden layer.
Neuron connected to region of
input. Only “sees” these values.

The Convolution Operation

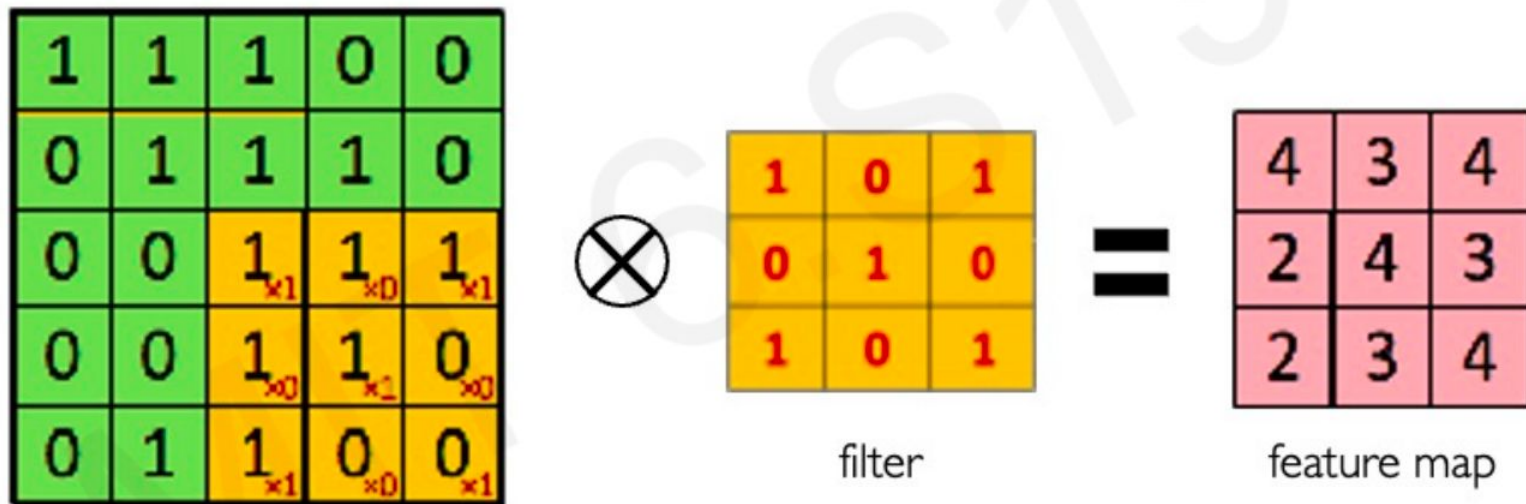
Suppose we want to compute the convolution of a 5x5 image and a 3x3 filter:



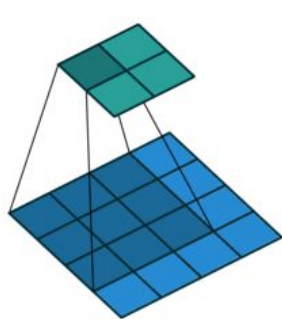
We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs...

The Convolution Operation

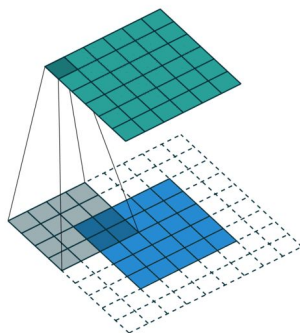
We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



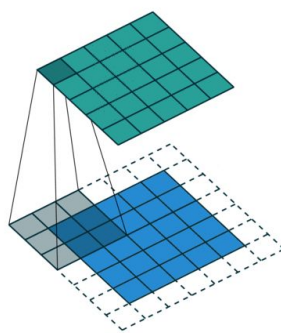
Padding and Stride



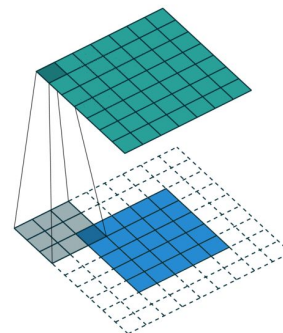
No padding,
no strides



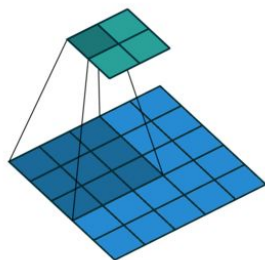
Arbitrary padding,
no strides



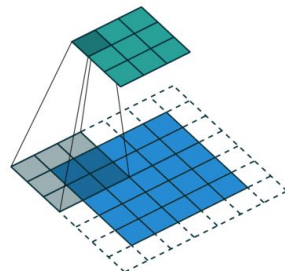
Half padding,
no strides



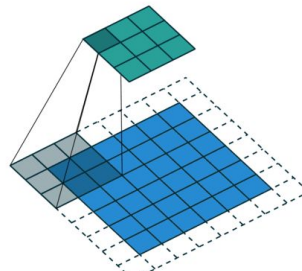
Full padding,
no strides



No padding,
strides



Padding,
strides



Padding,
Strides (odd)

Output feature dimension

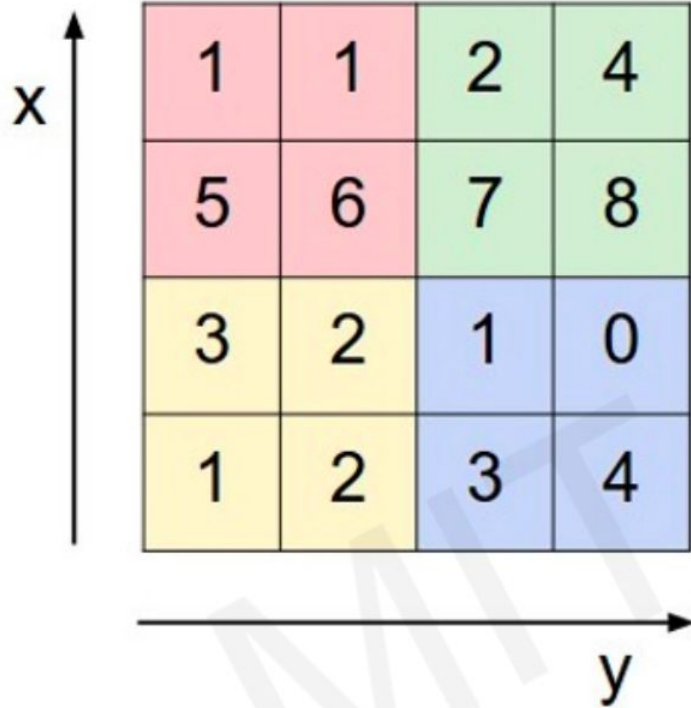
- Consider that we have an input feature map with dimensions $W \times W \times D$
- We also have D' filters/kernels each with dimensions $F \times F$, padding P and stride S
- What will be the dimension of the output feature map W_{out} ?

$$W_{out} = \frac{W - F + 2P}{S} + 1$$

<https://towardsdatascience.com/convolutional-neural-networks-explained-9cc5188c4939>

- Additionally, since there are D' filters that are being applied, the final output dimension is $W_{out} \times W_{out} \times D'$

Pooling



1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

max pool with 2x2 filters
and stride 2



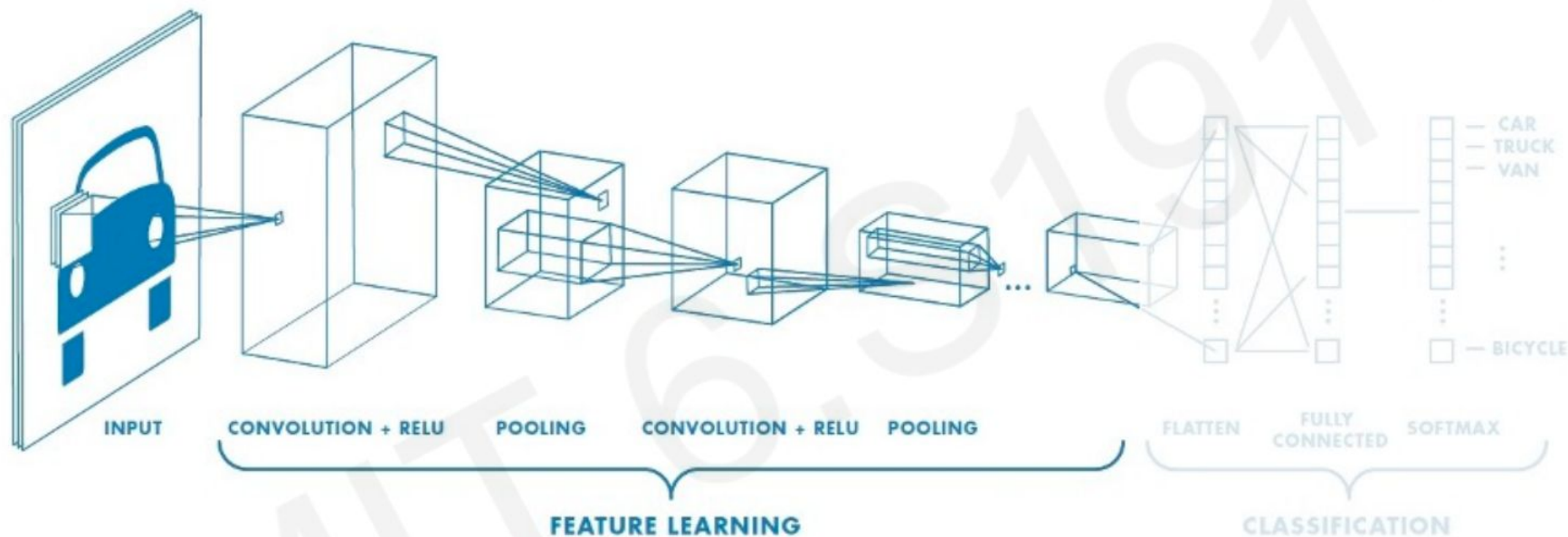
6	8
3	4

- 1) Reduced dimensionality
- 2) Spatial invariance

How else can we downsample and preserve spatial invariance?

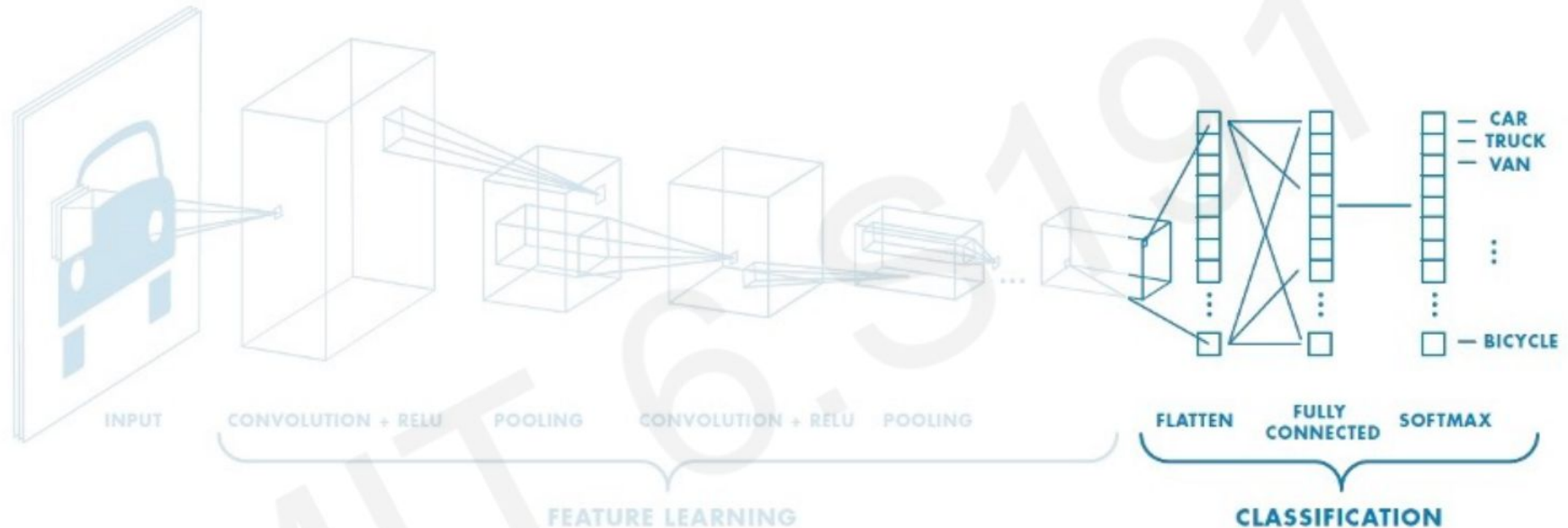
Average Pooling!

CNNs for Classification: Feature Learning



1. Learn features in input image through **convolution**
2. Introduce **non-linearity** through activation function (real-world data is non-linear!)
3. Reduce dimensionality and preserve spatial invariance with **pooling**

CNNs for Classification: Class Probabilities

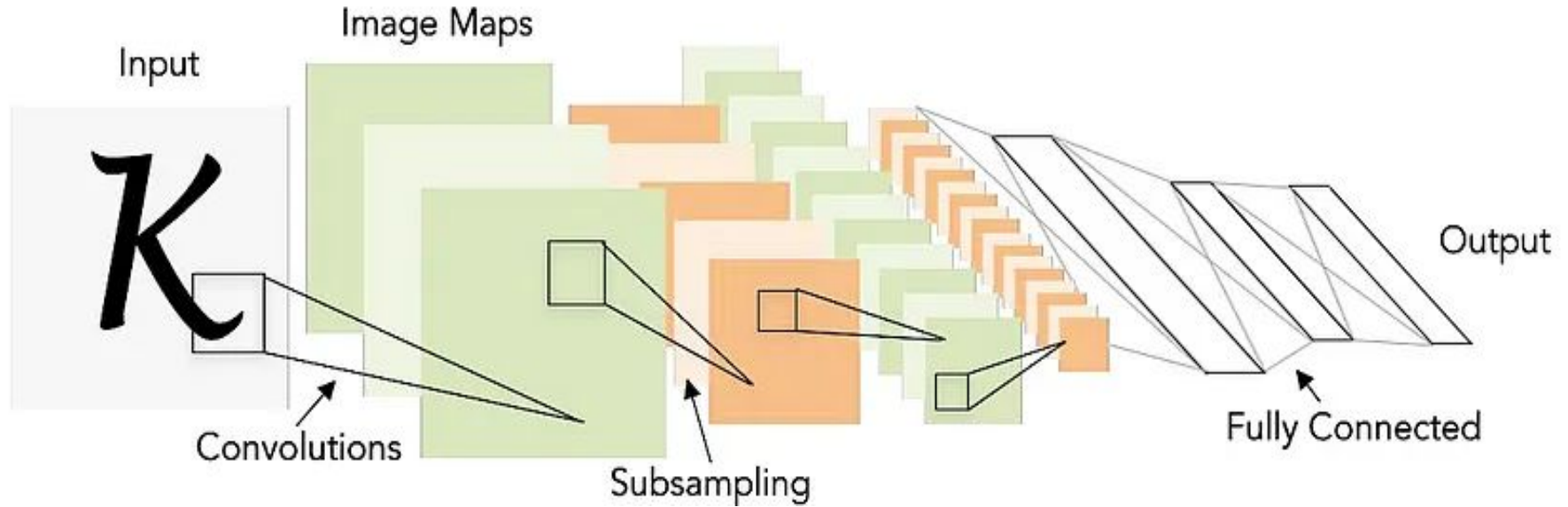


- CONV and POOL layers output high-level features of input
- Fully connected layer uses these features for classifying input image
- Express output as **probability** of image belonging to a particular class

$$\text{softmax}(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}}$$

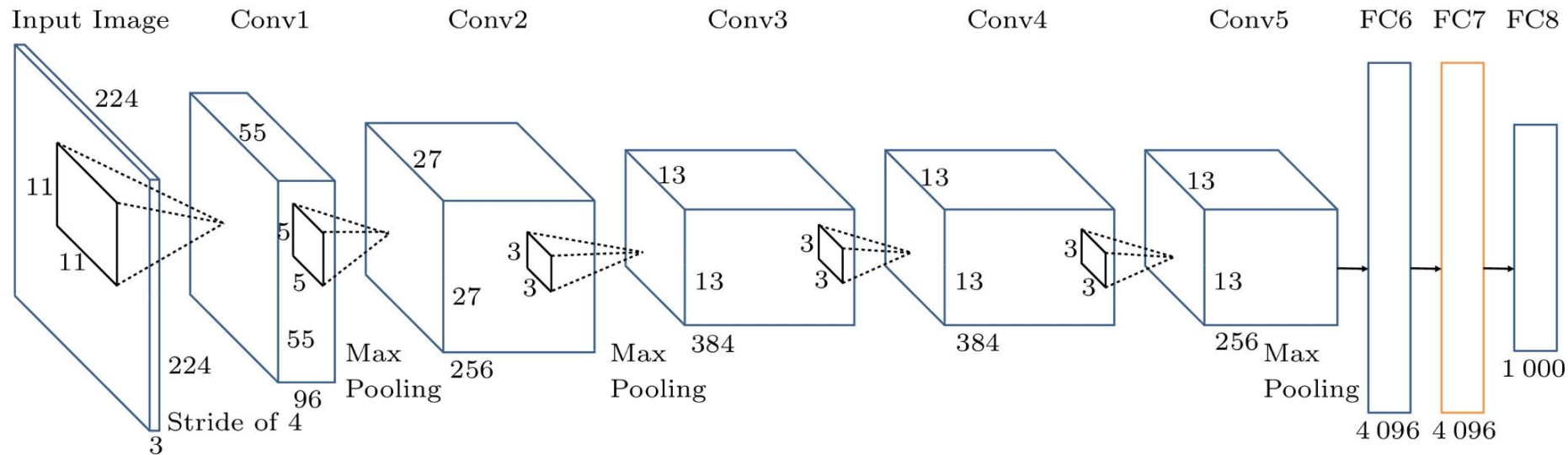
Brief history of various CNN architectures

LeNet-5 (1989-1998)

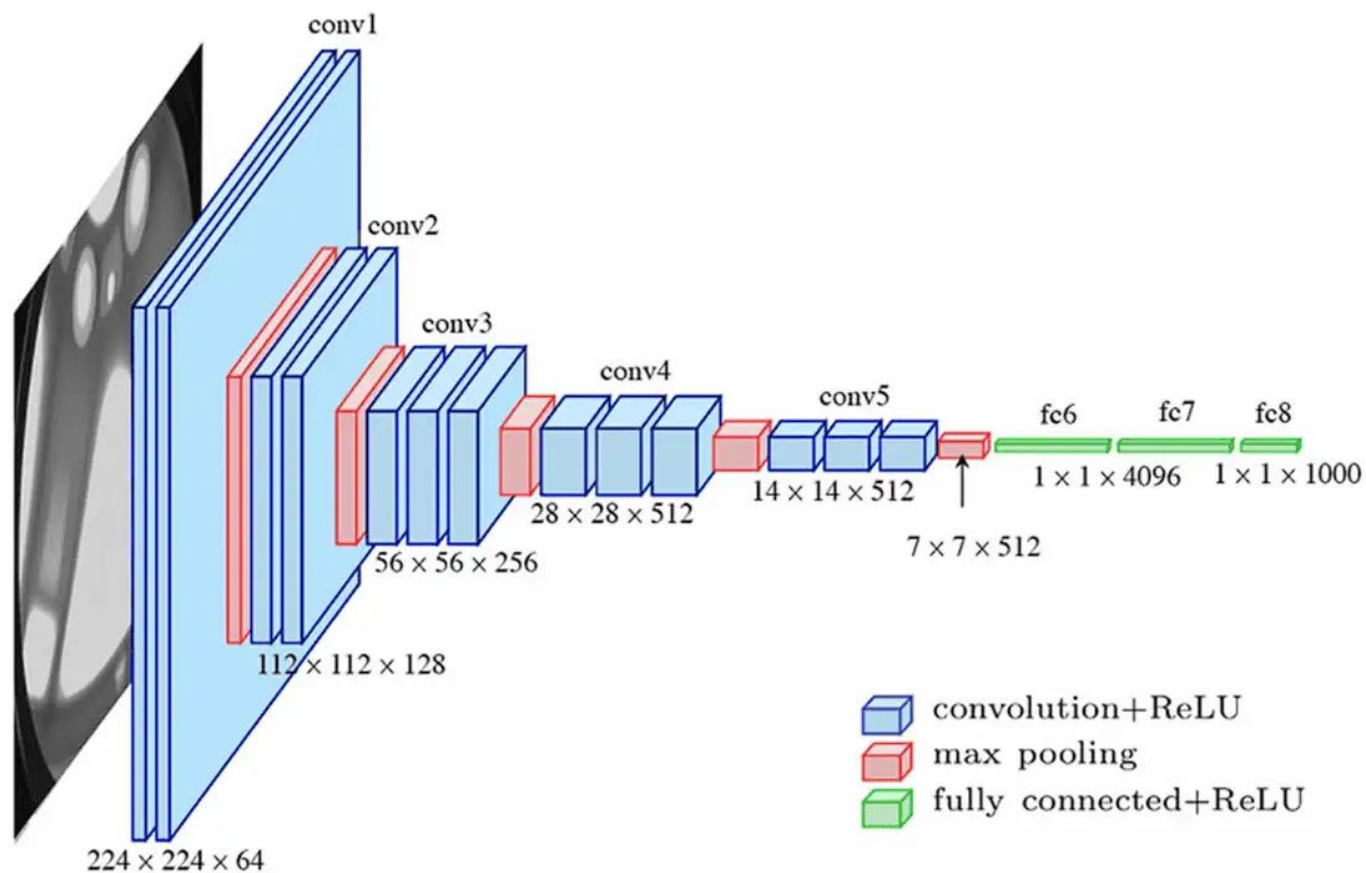


<https://medium.com/appyhigh-technology-blog/convolutional-neural-networks-a-brief-history-of-their-evolution-ee3405568597>

AlexNet (2012)



VGGNet (2014)

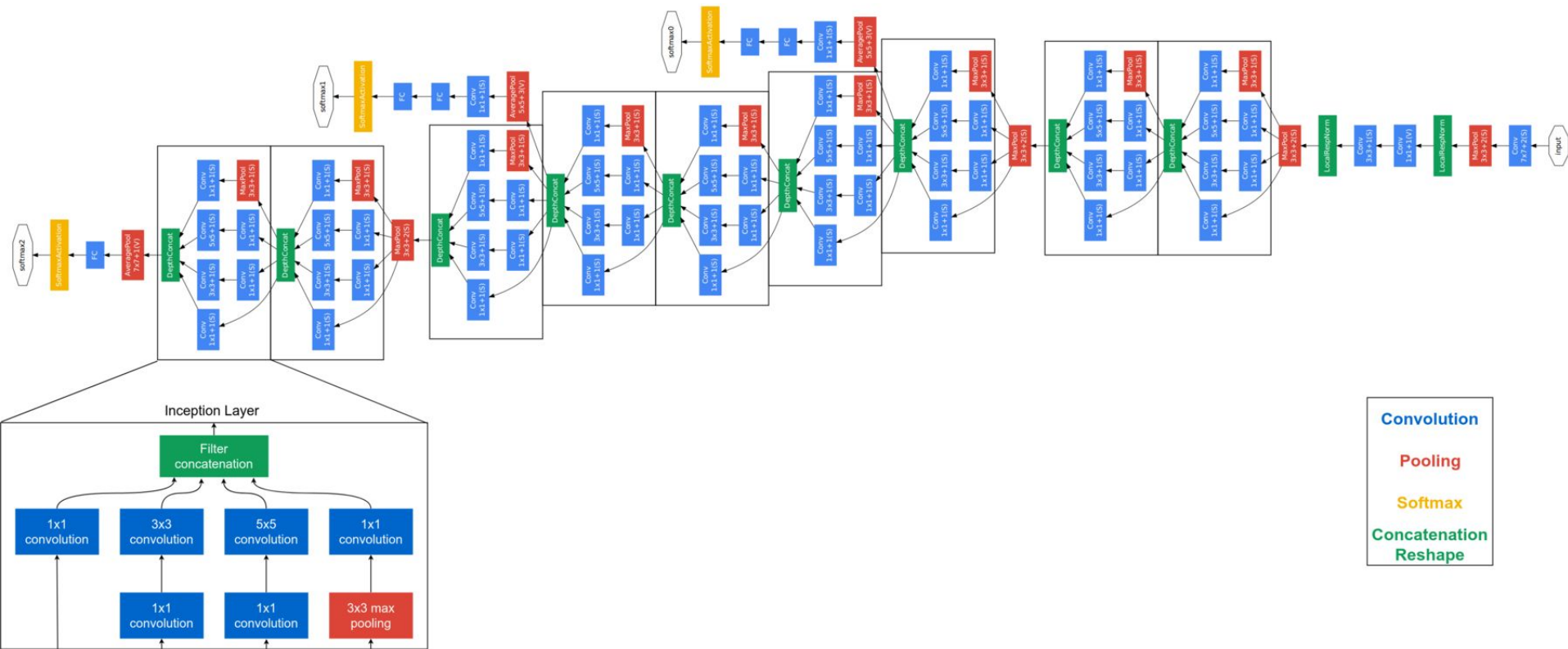


A close-up shot of Leonardo DiCaprio in a dark suit and tie, looking slightly to his right with a serious expression. Another man's profile is visible on the right side of the frame, looking towards DiCaprio. The background is dark and out of focus.

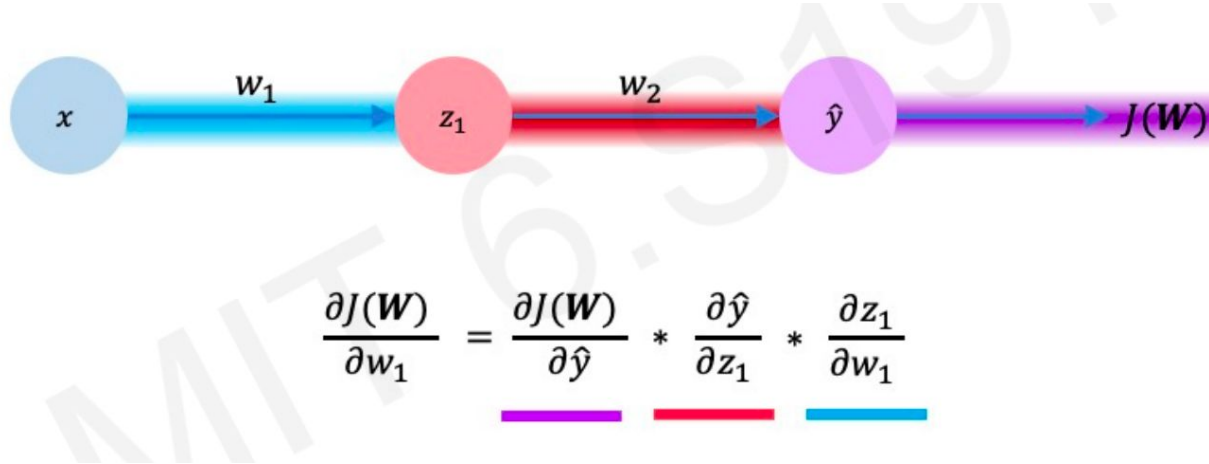
WE NEED TO GO

DEEPER

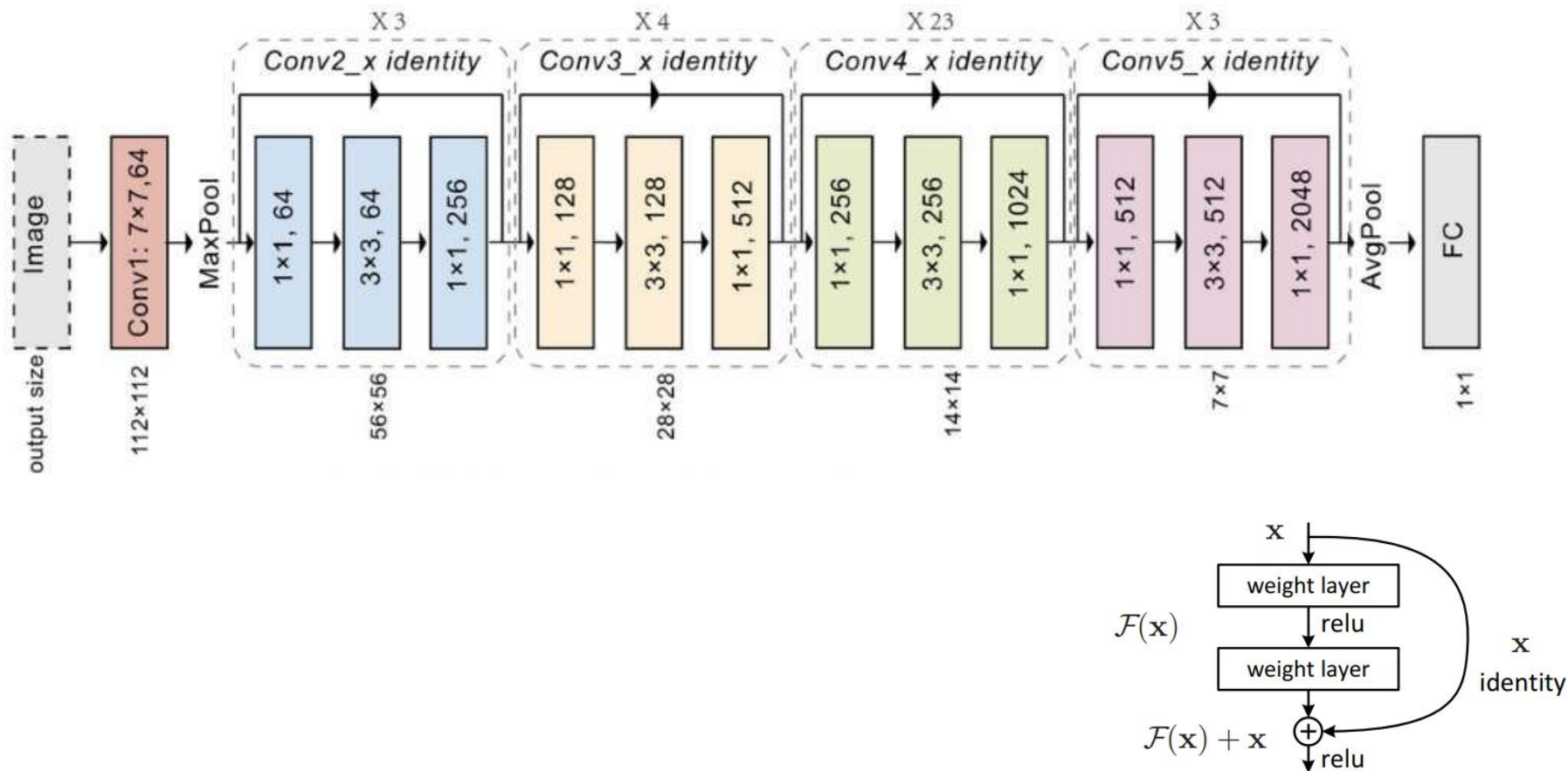
GoogleNet (2014)



Vanishing/Exploding Gradients



ResNet (2015)

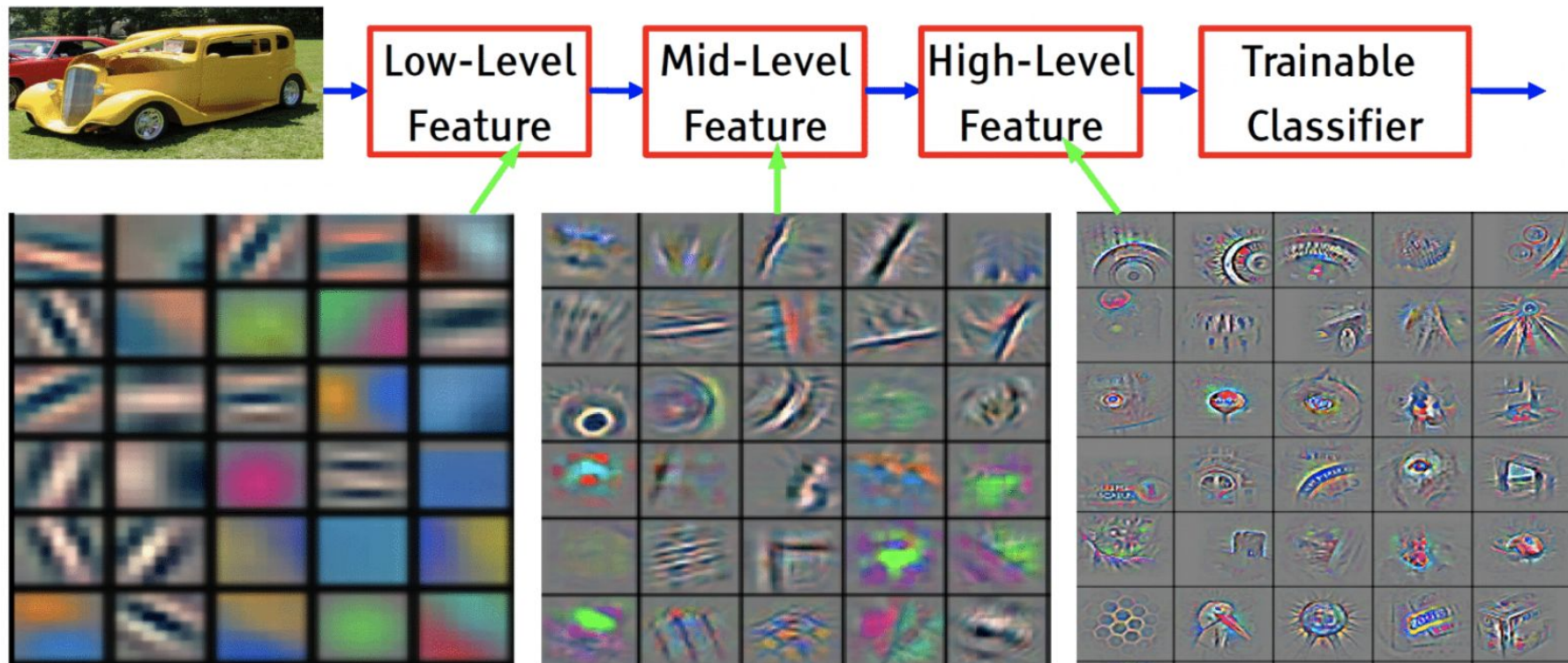


Transfer Learning and Fine-tuning

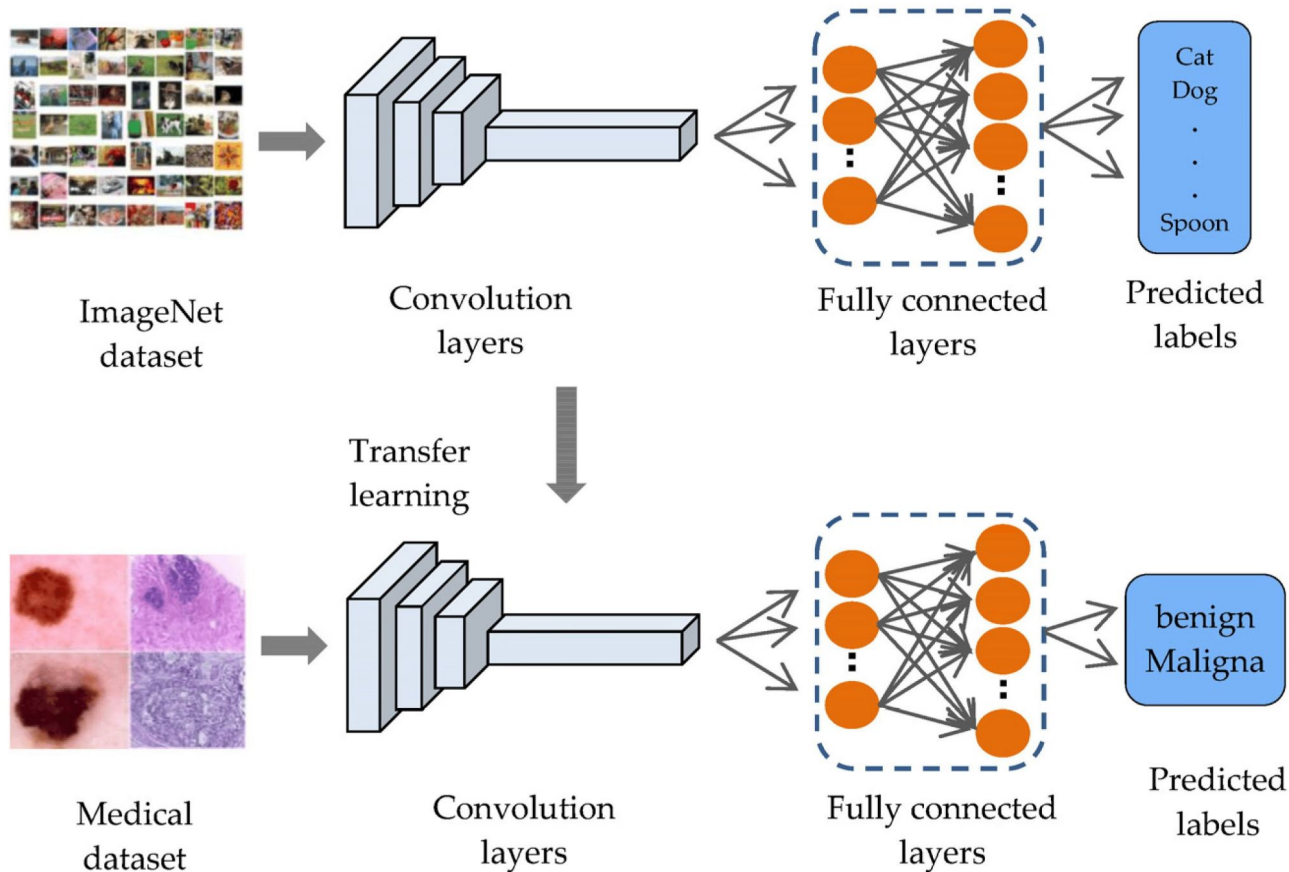
ImageNet



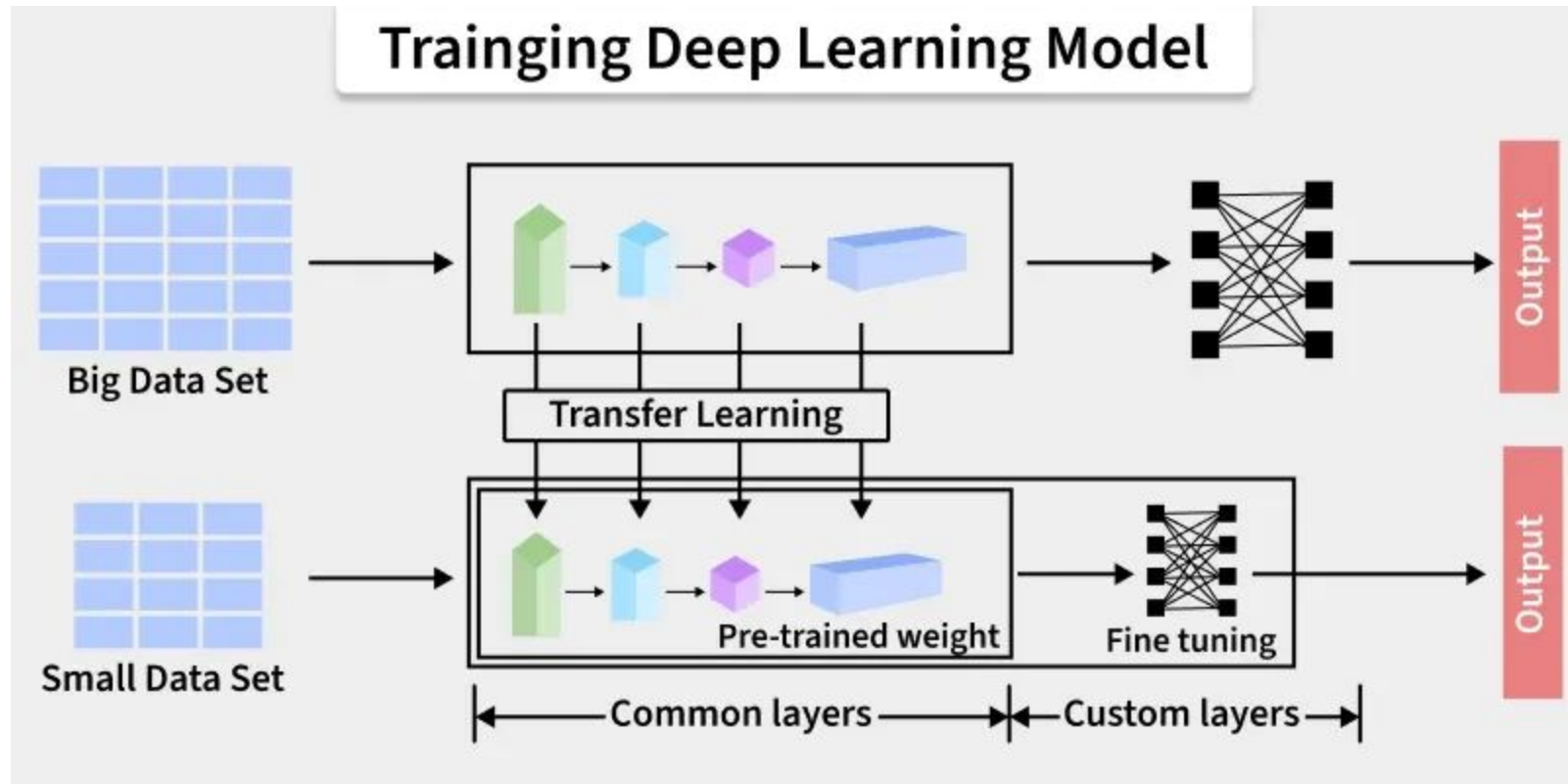
Feature Hierarchy of Pretrained CNNs



Transfer Learning



Fine-tuning

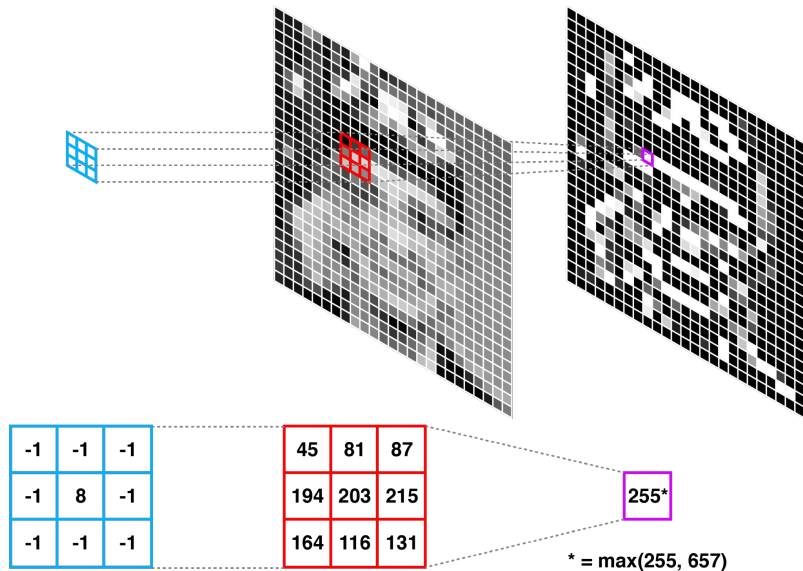


Catastrophic forgetting

- Unfreezing more layers
- Higher learning rates
- Similar/small datasets

Learning for Sequential Data

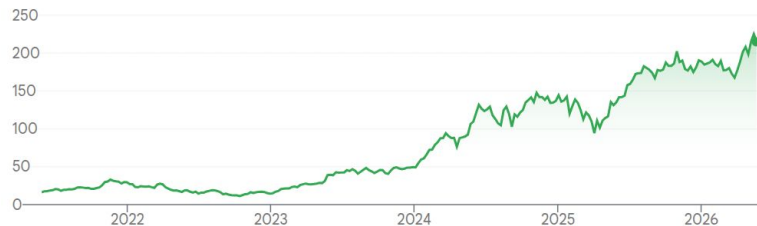
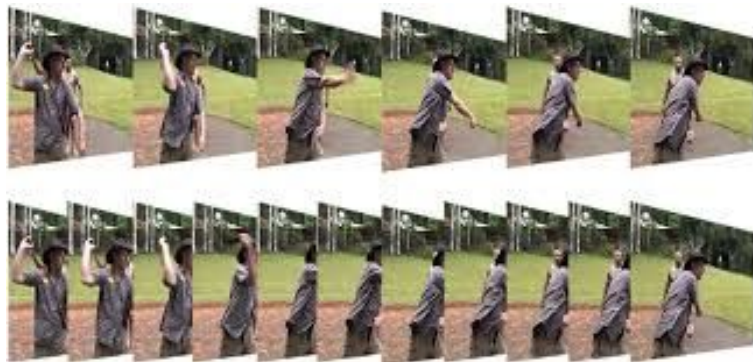
Beyond CNNs: Memory, Attention & Self-Supervision



grew a pretty little fir-tree; and yet it was not happy

"Rejoice with us," said the air and the sunlight. Enjoy

The sun shone, and the soft air fluttered its leaves

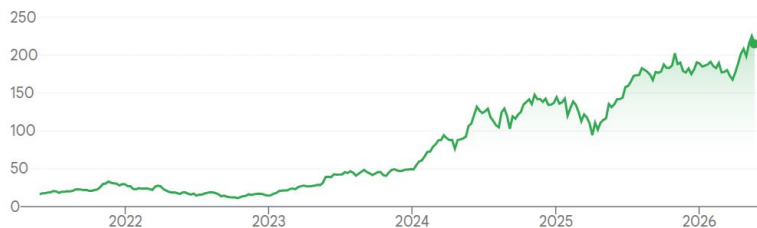


Why CNNs are not enough

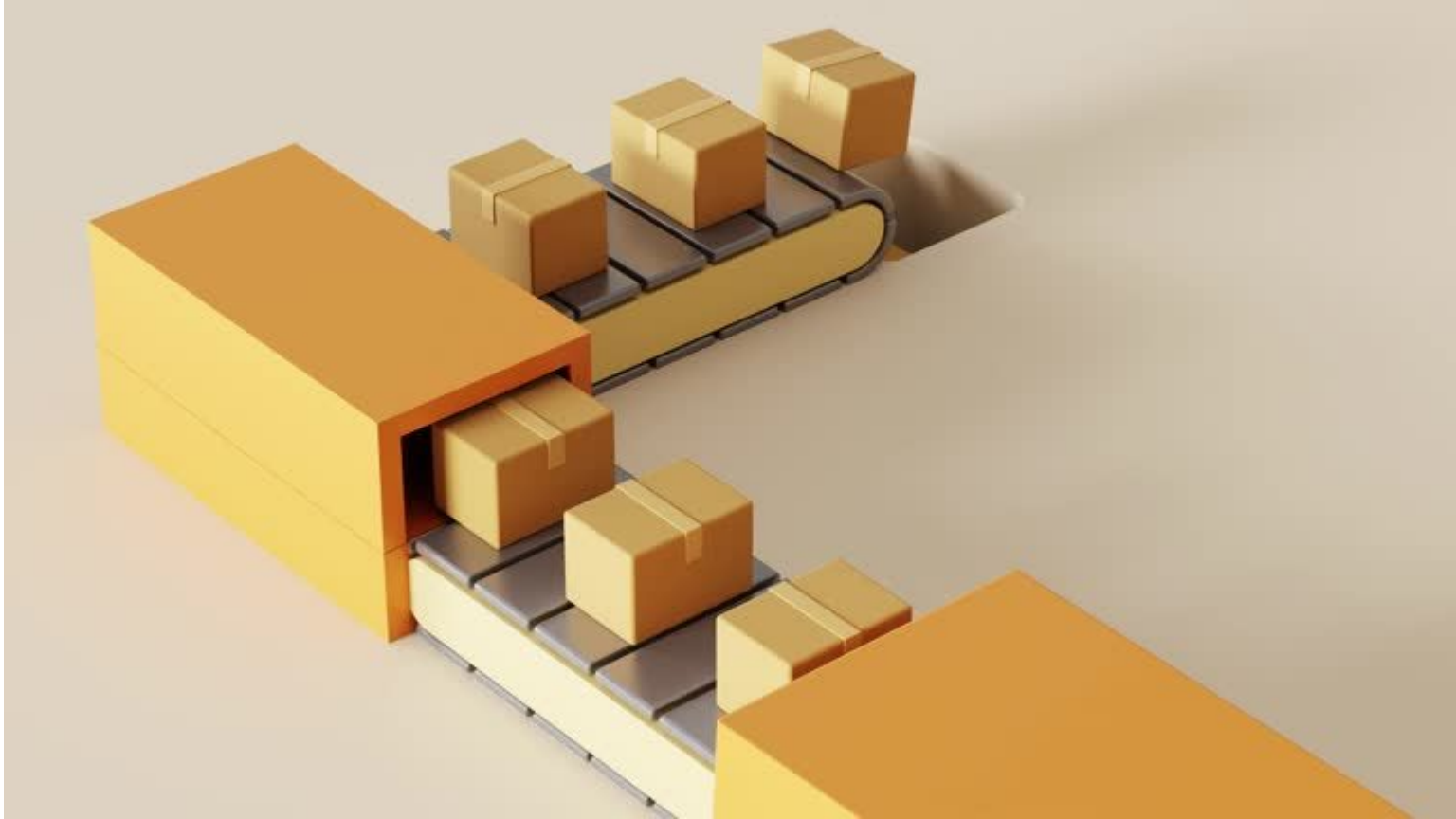
Man bit dog

v/s

Dog bit man

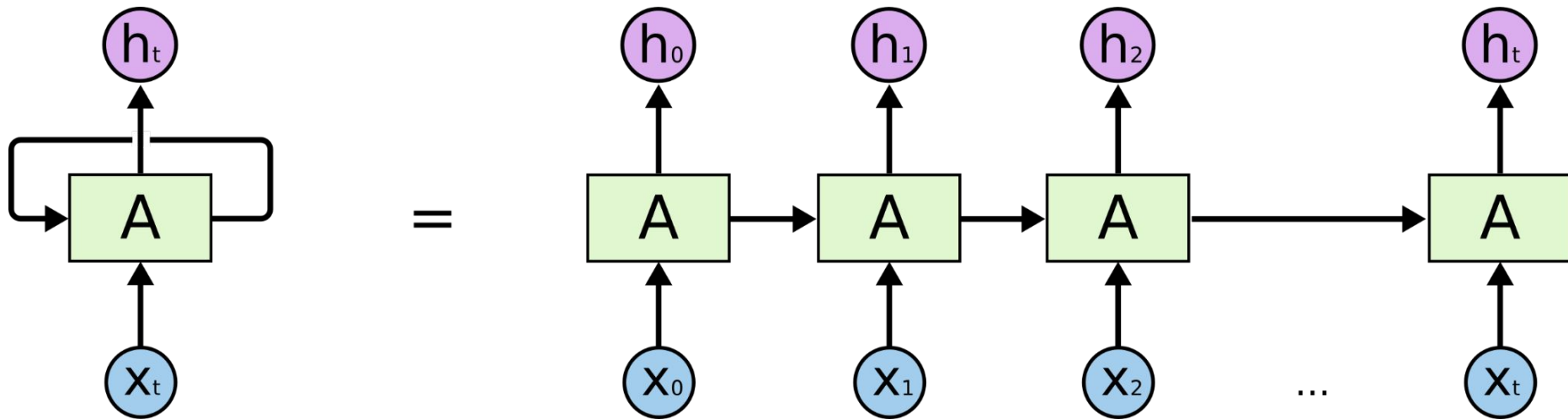


Need for Memory – Hidden States



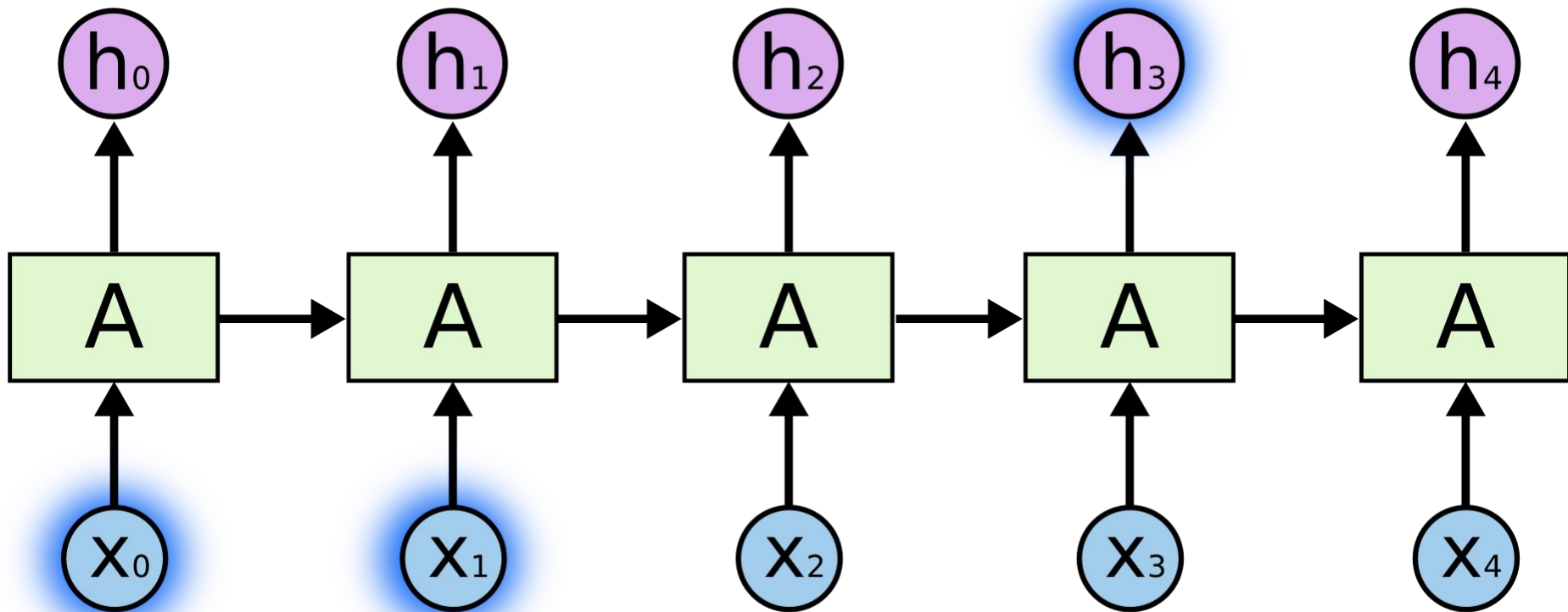
Recurrent Neural Networks – RNNs

What is an RNN?

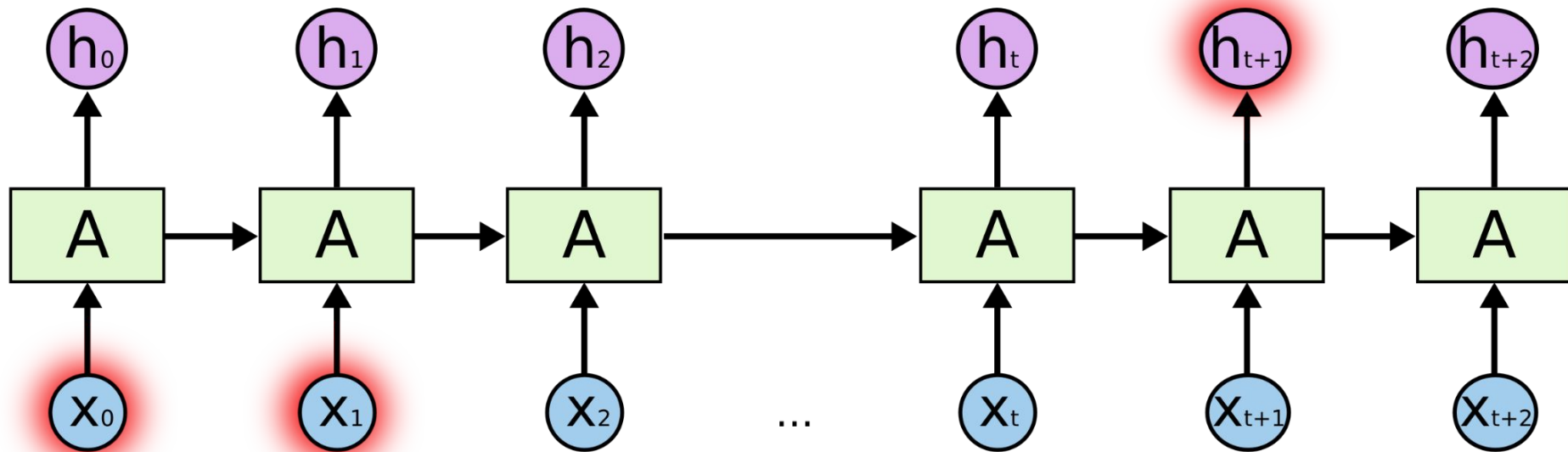


$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

Why recurrence is useful



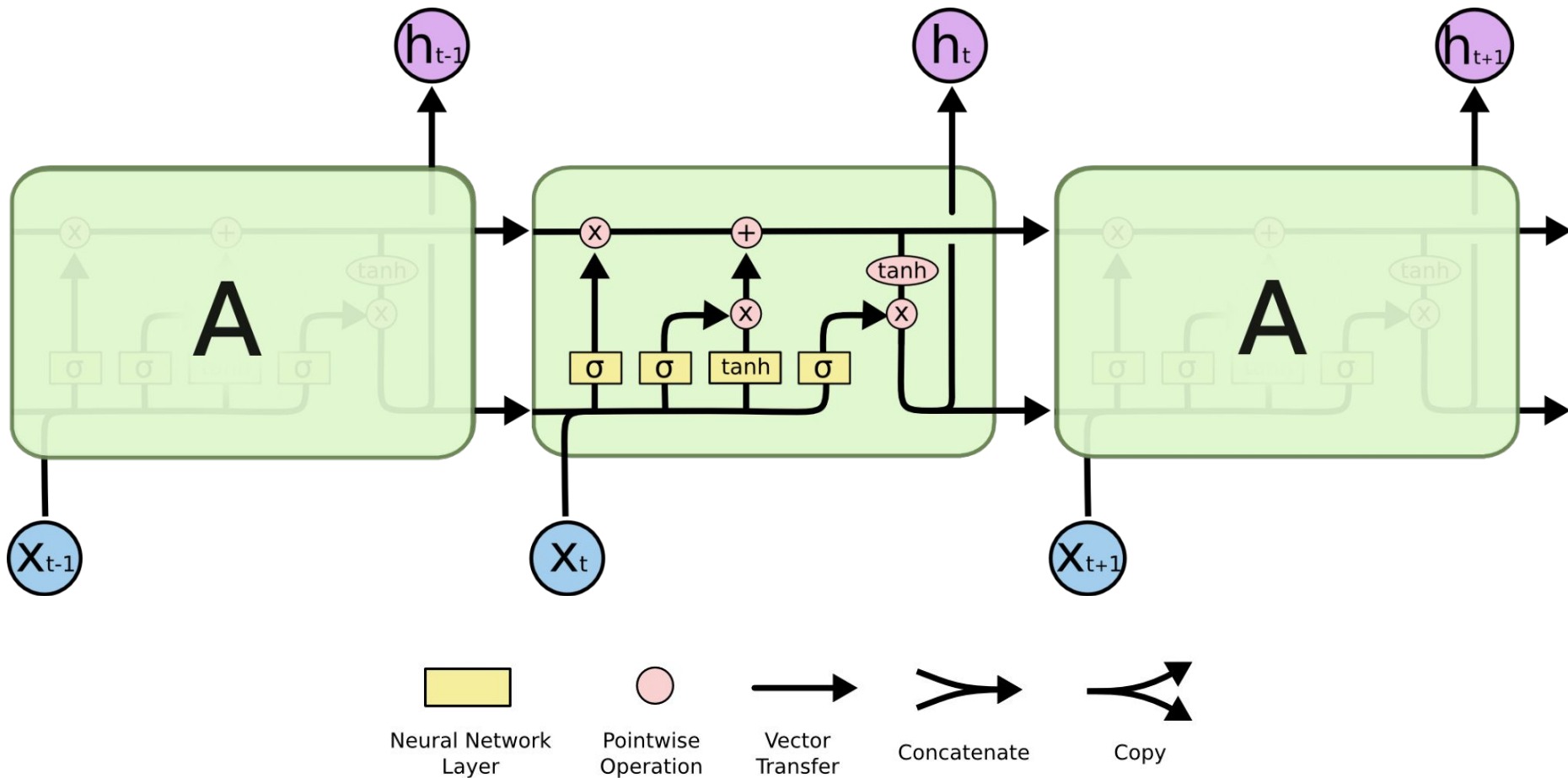
Problem with vanilla RNNs



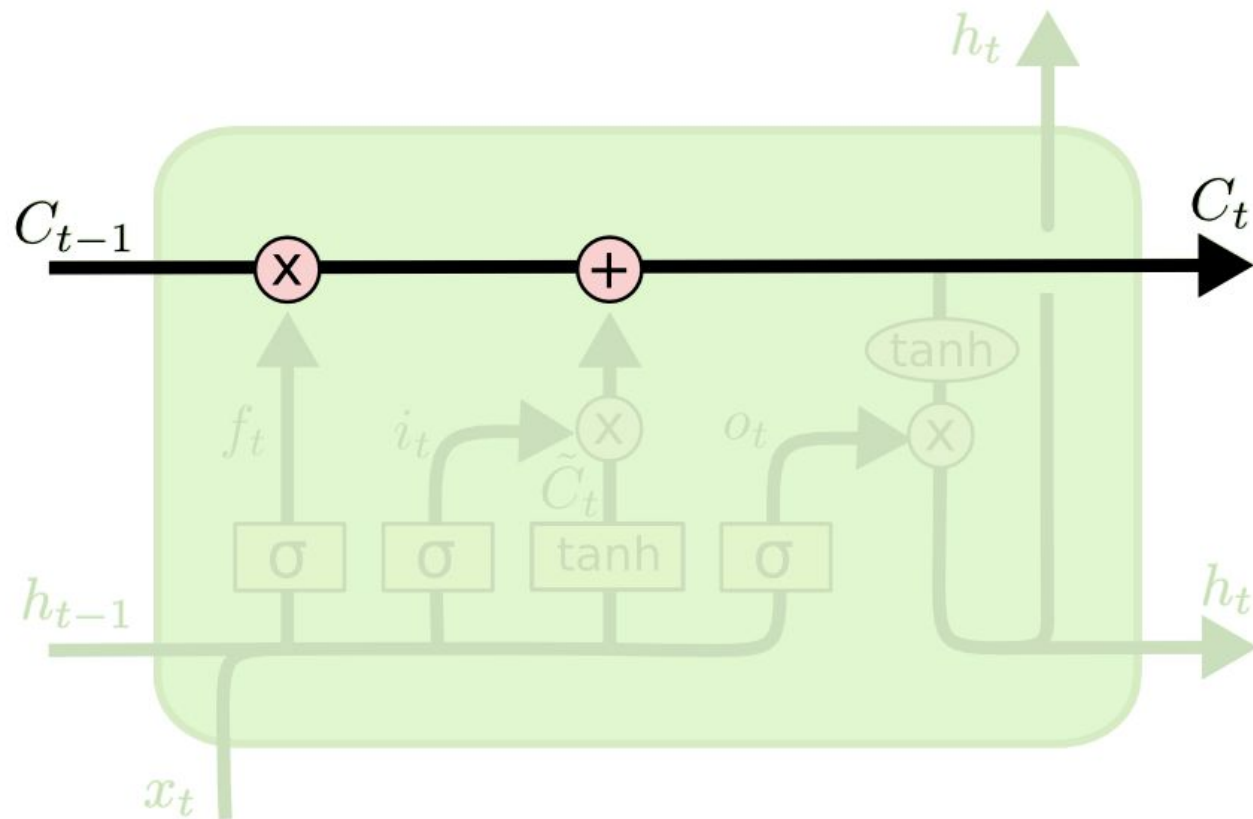
Nobody expects the Vanishing gradients inquisition!

LSTM – Long Short-Term Memory

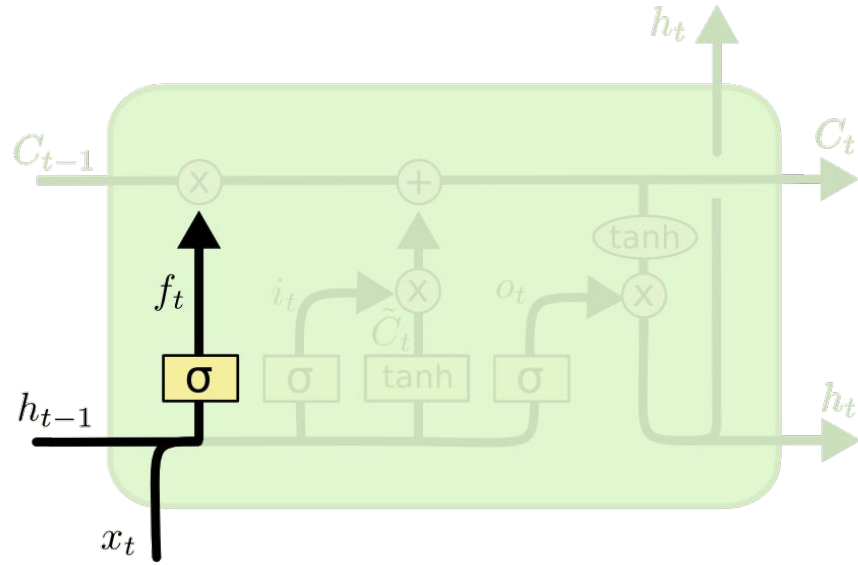
Why LSTMs were needed



LSTM – Cell State

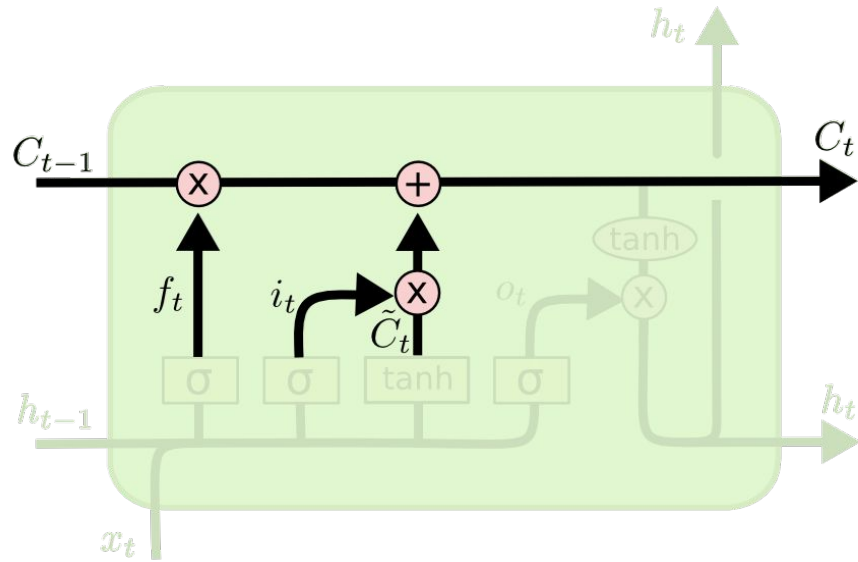


LSTM – Forget Gate



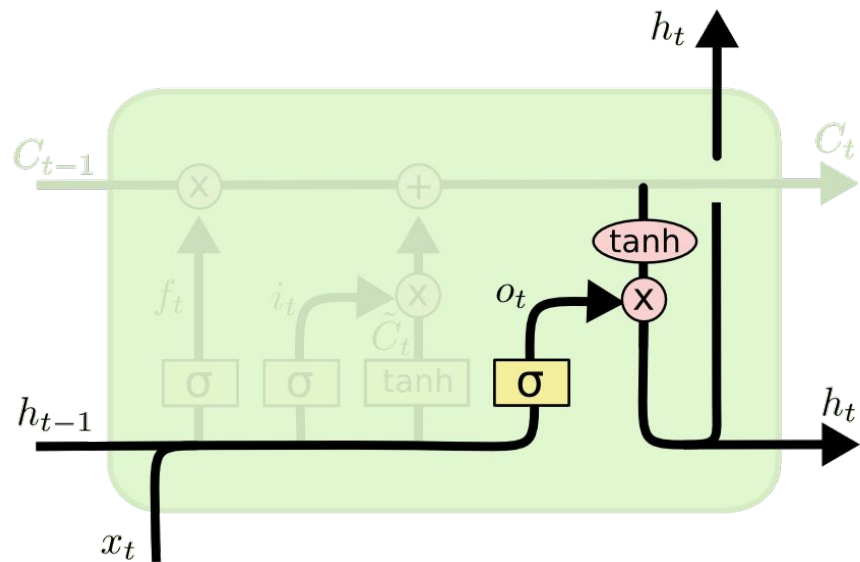
$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

LSTM – Cell State update



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

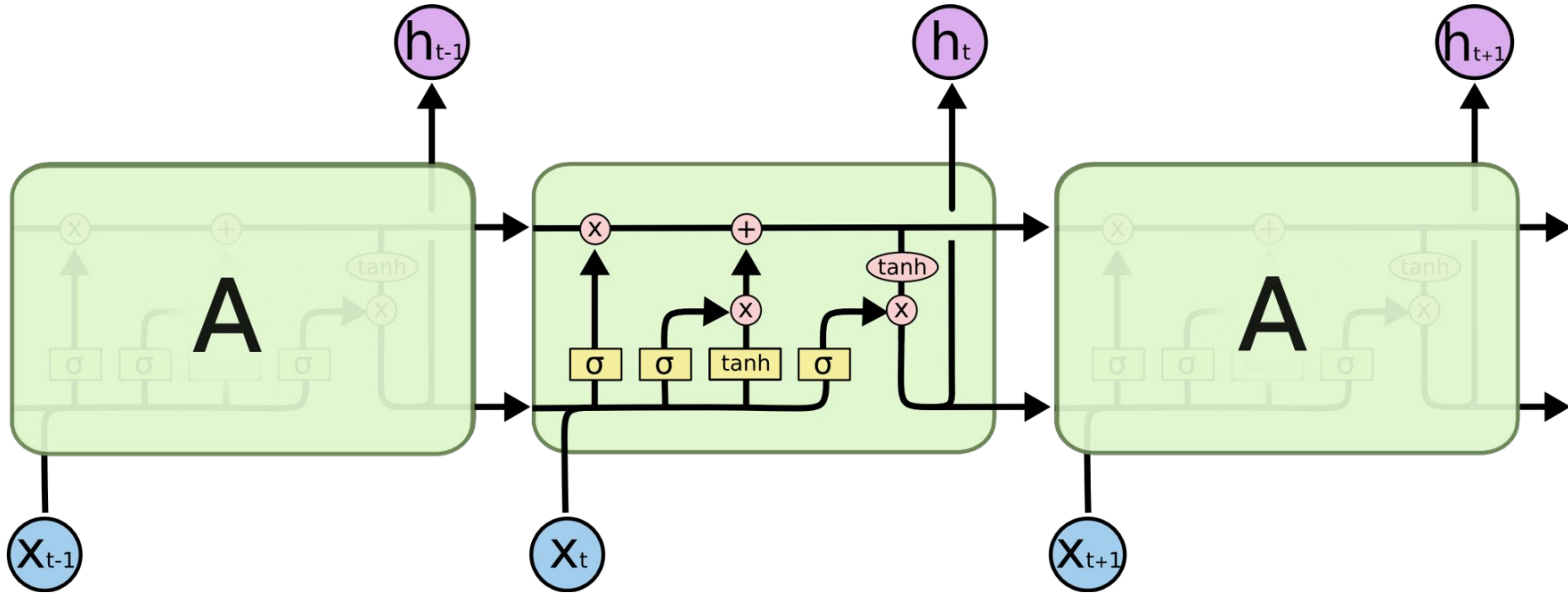
LSTM – Output Gate



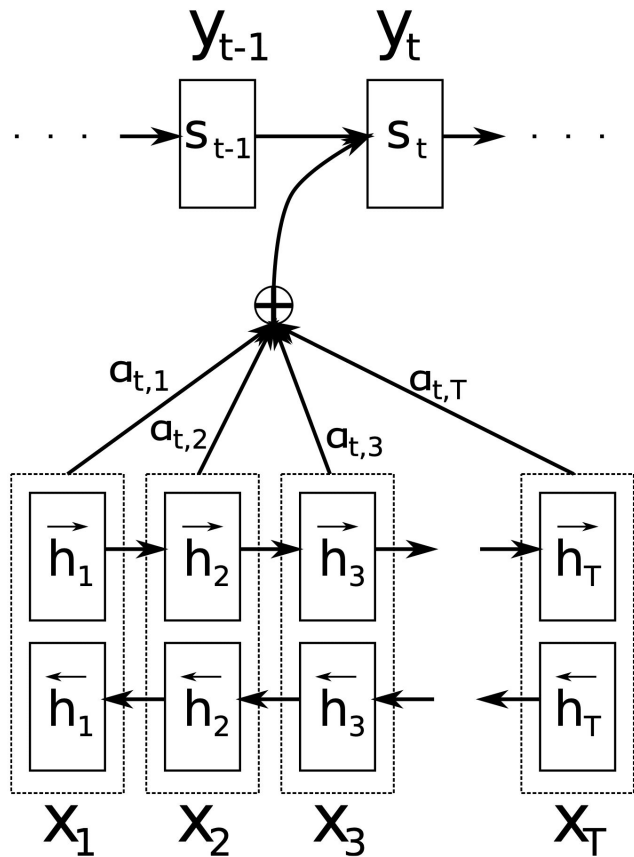
$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

The full LSTM block



Bahdanau Attention



$$\alpha_{ts} = \frac{\exp(\text{score}(\vec{h}_t, \vec{h}_s))}{\sum_{s'=1}^S \exp(\text{score}(\vec{h}_t, \vec{h}_{s'}))}$$

[Attention weights]

$$\vec{c}_t = \sum_s \alpha_{ts} \vec{h}_s$$

[Context vector]

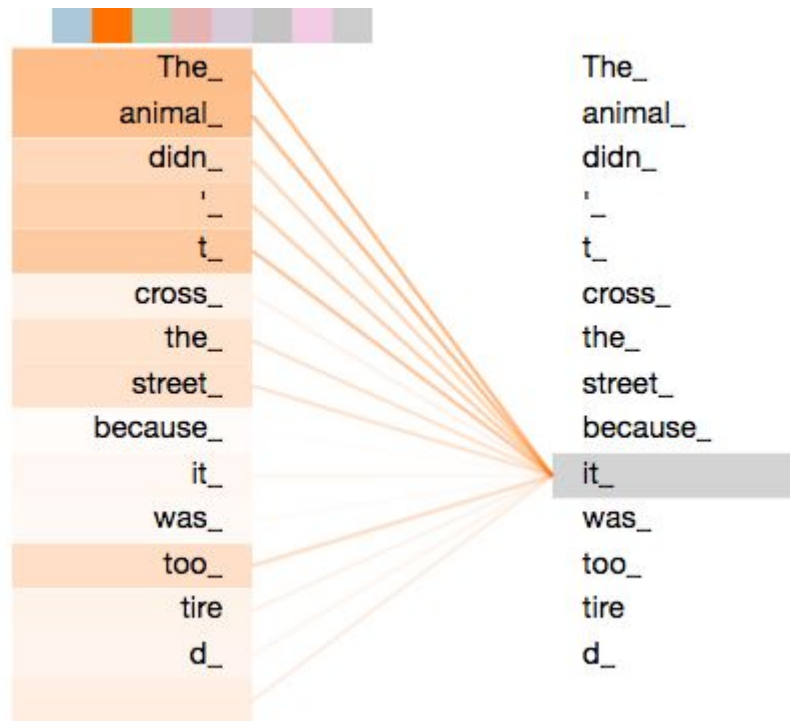
$$\vec{a}_t = f(\vec{c}_t, \vec{h}_t) = \tanh(\mathbf{W}_c[\vec{c}_t; \vec{h}_t])$$

[Attention vector]

$$\text{score}(\vec{h}_t, \vec{h}_s) = \begin{cases} \vec{h}_t^\top \mathbf{W} \vec{h}_s & \text{[Luong's multiplicative style]} \\ \vec{v}_a^\top \tanh(\mathbf{W}_1 \vec{h}_t + \mathbf{W}_2 \vec{h}_s) & \text{[Bahdanau's additive style]} \end{cases}$$

Transformers

Self-Attention



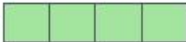
Self-Attention

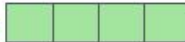
Input

Thinking

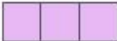
Machines

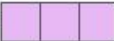
Embedding

x_1 

x_2 

Queries

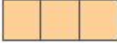
q_1 

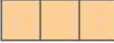
q_2 



W^Q

Keys

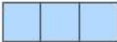
k_1 

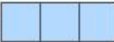
k_2 



W^K

Values

v_1 

v_2 



W^V

Self-Attention

Input

Embedding

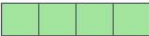
Queries

Keys

Values

Score

Thinking

x_1 

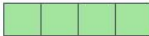
q_1 

k_1 

v_1 

$q_1 \cdot k_1 = 112$

Machines

x_2 

q_2 

k_2 

v_2 

$q_1 \cdot k_2 = 96$

Self-Attention

Input

Embedding

Queries

Keys

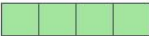
Values

Score

Divide by 8 ($\sqrt{d_k}$)

Softmax

Thinking

x_1 

q_1 

k_1 

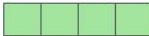
v_1 

$q_1 \cdot k_1 = 112$

14

0.88

Machines

x_2 

q_2 

k_2 

v_2 

$q_1 \cdot k_2 = 96$

12

0.12

Self-Attention

Input

Embedding

Queries

Keys

Values

Score

Divide by 8 ($\sqrt{d_k}$)

Softmax

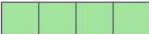
Softmax

X

Value

Sum

Thinking

x_1 

q_1 

k_1 

v_1 

$q_1 \cdot k_1 = 112$

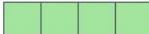
14

0.88

v_1 

z_1 

Machines

x_2 

q_2 

k_2 

v_2 

$q_1 \cdot k_2 = 96$

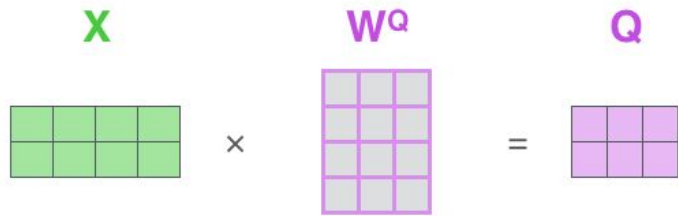
12

0.12

v_2 

z_2 

Self-Attention – Matrix Formulation

$$\mathbf{X} \times \mathbf{W}^Q = \mathbf{Q}$$


The diagram illustrates the calculation of the Query matrix $\mathbf{Q}$. It shows a green input matrix $\mathbf{X}$ (2 rows, 4 columns) multiplied by a purple weight matrix $\mathbf{W}^Q$ (4 rows, 3 columns) to produce a purple output matrix $\mathbf{Q}$ (2 rows, 3 columns).

$$\mathbf{X} \times \mathbf{W}^K = \mathbf{K}$$


The diagram illustrates the calculation of the Key matrix $\mathbf{K}$. It shows a green input matrix $\mathbf{X}$ (2 rows, 4 columns) multiplied by an orange weight matrix $\mathbf{W}^K$ (4 rows, 3 columns) to produce an orange output matrix $\mathbf{K}$ (2 rows, 3 columns).

$$\mathbf{X} \times \mathbf{W}^V = \mathbf{V}$$


The diagram illustrates the calculation of the Value matrix $\mathbf{V}$. It shows a green input matrix $\mathbf{X}$ (2 rows, 4 columns) multiplied by a blue weight matrix $\mathbf{W}^V$ (4 rows, 3 columns) to produce a blue output matrix $\mathbf{V}$ (2 rows, 3 columns).

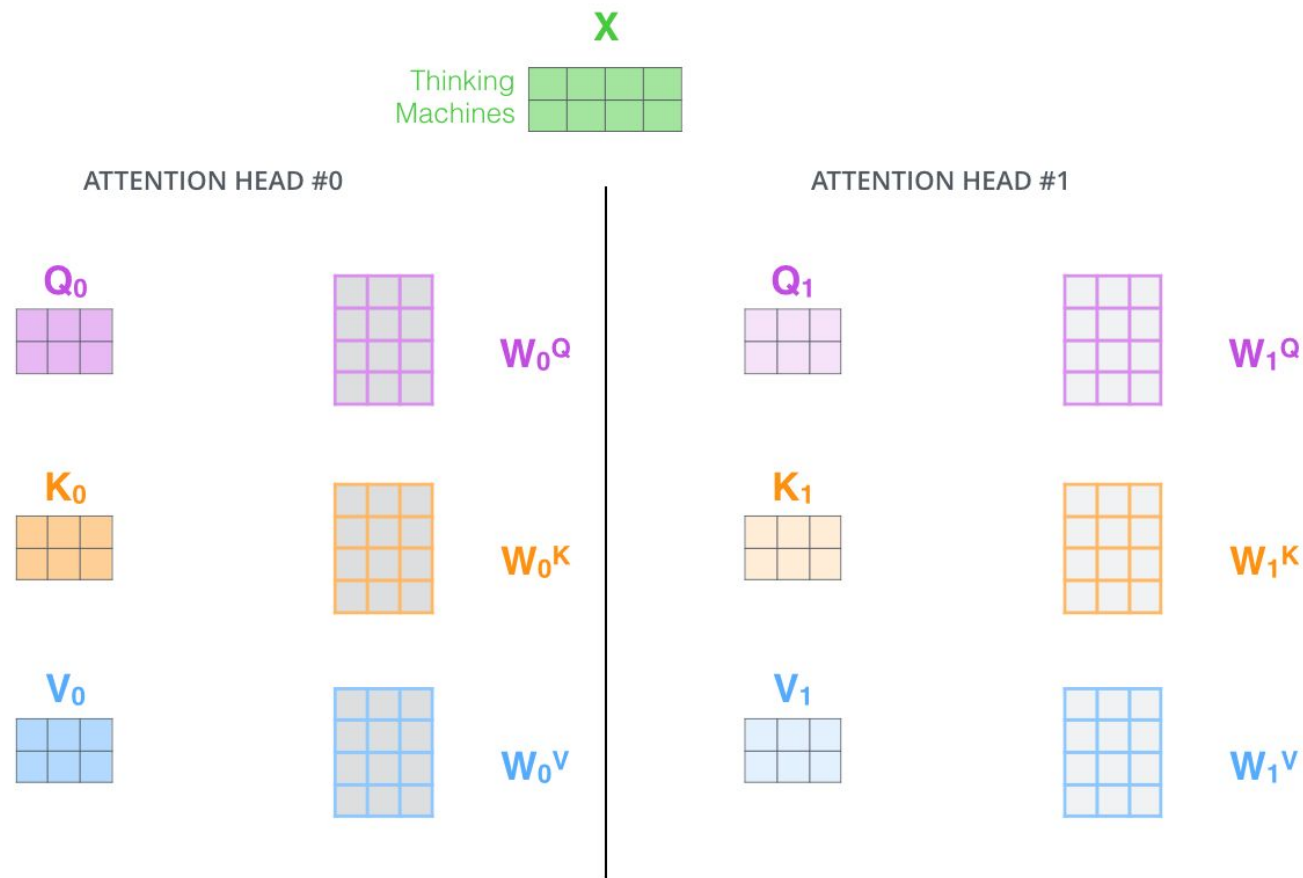
Self-Attention – Matrix Formulation

$$\text{softmax}\left(\frac{\overset{\text{Q}}{\begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array}} \times \overset{\text{K}^T}{\begin{array}{|c|c|} \hline & \\ \hline & \\ \hline & \\ \hline \end{array}}}{\sqrt{d_k}}\right) \overset{\text{V}}{\begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array}}$$

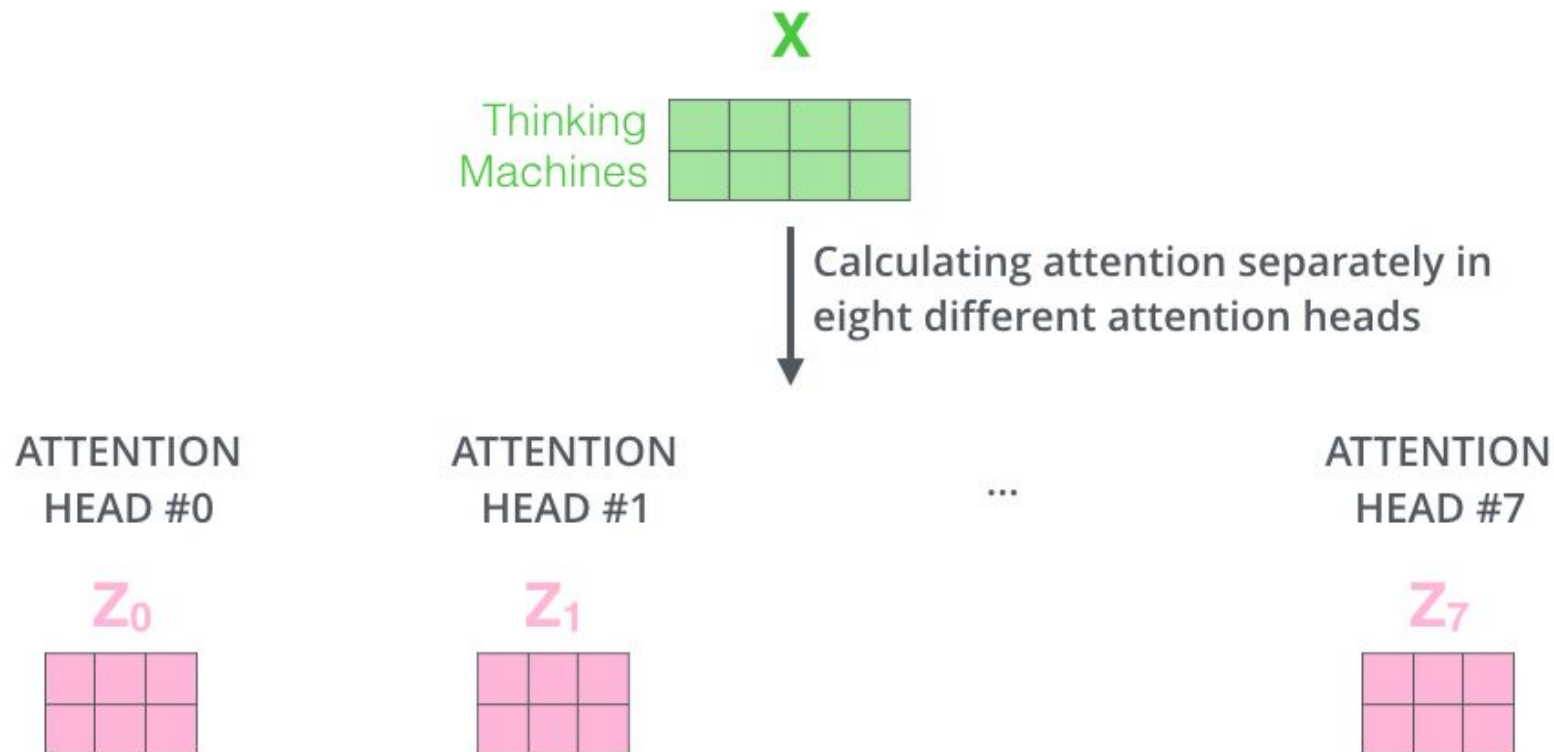
=

$\overset{\text{Z}}{\begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array}}$

Multi-headed Self-Attention



Multi-headed Self-Attention



Multi-headed Self-Attention

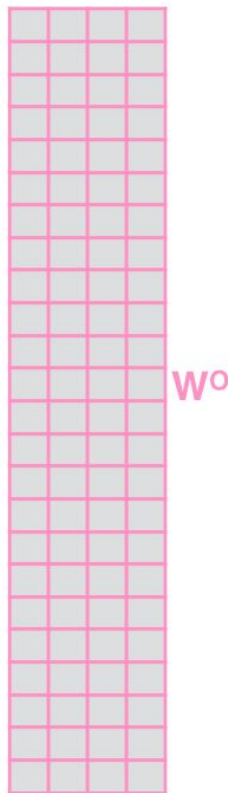
1) Concatenate all the attention heads



2) Multiply with a weight matrix W^O that was trained jointly with the model

$\times$

3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



Output of a single Attention block

1) This is our input sentence*

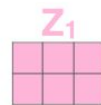
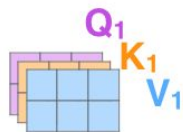
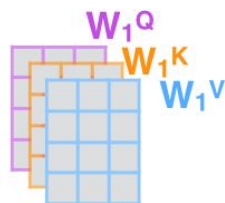
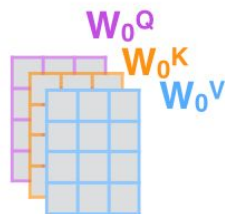
2) We embed each word*

3) Split into 8 heads. We multiply X or R with weight matrices

4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

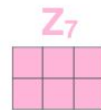
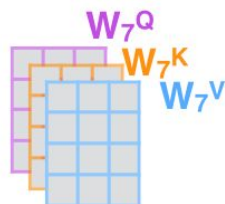
Thinking
Machines



...

...

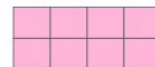
...



W^O

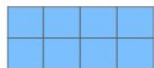


Z

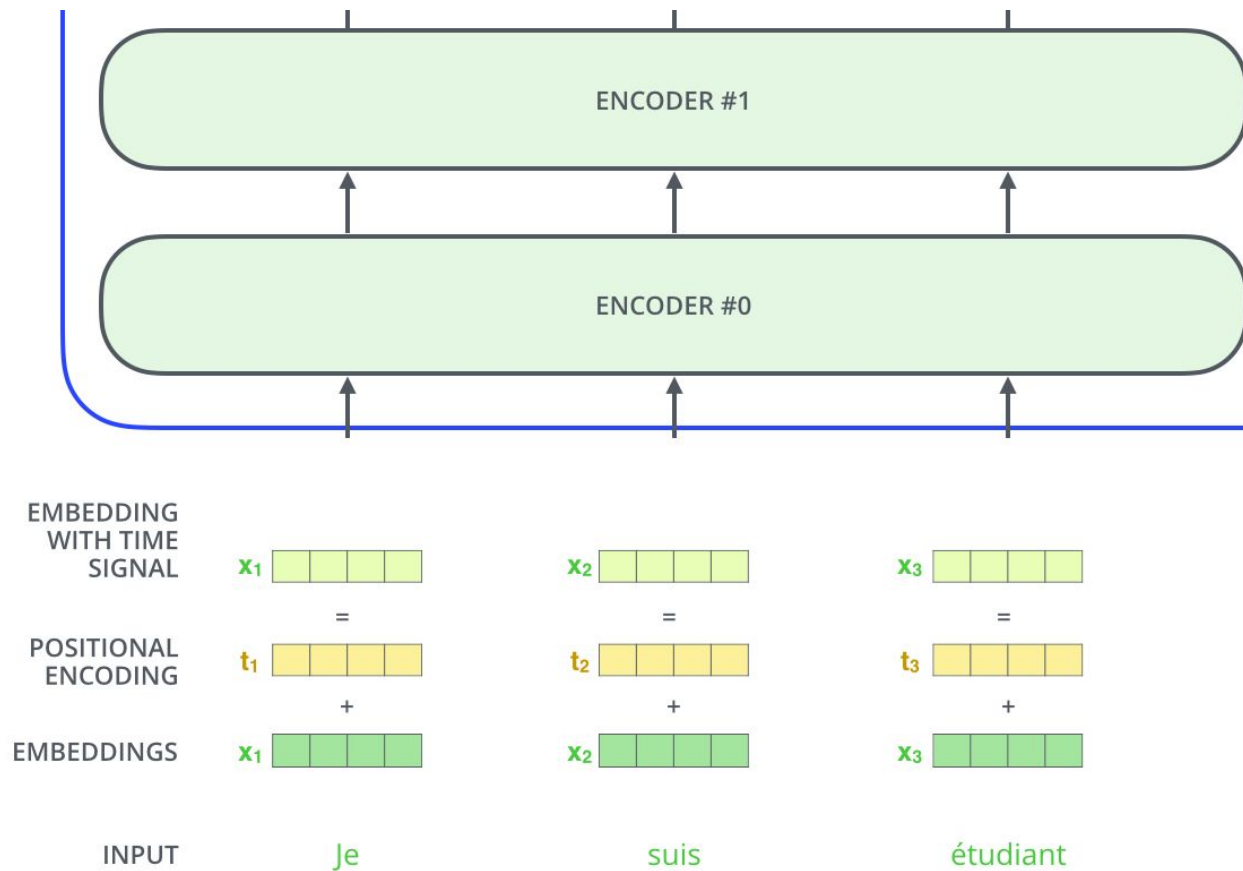


* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

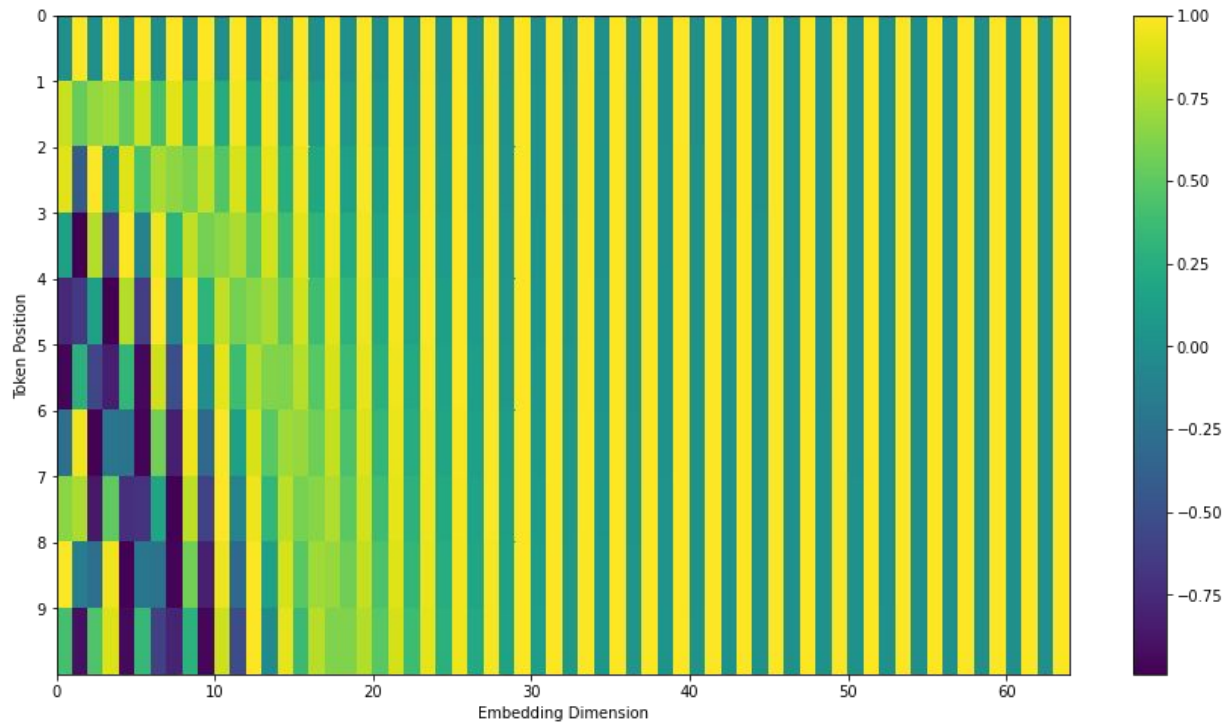
R



Positional Encoding

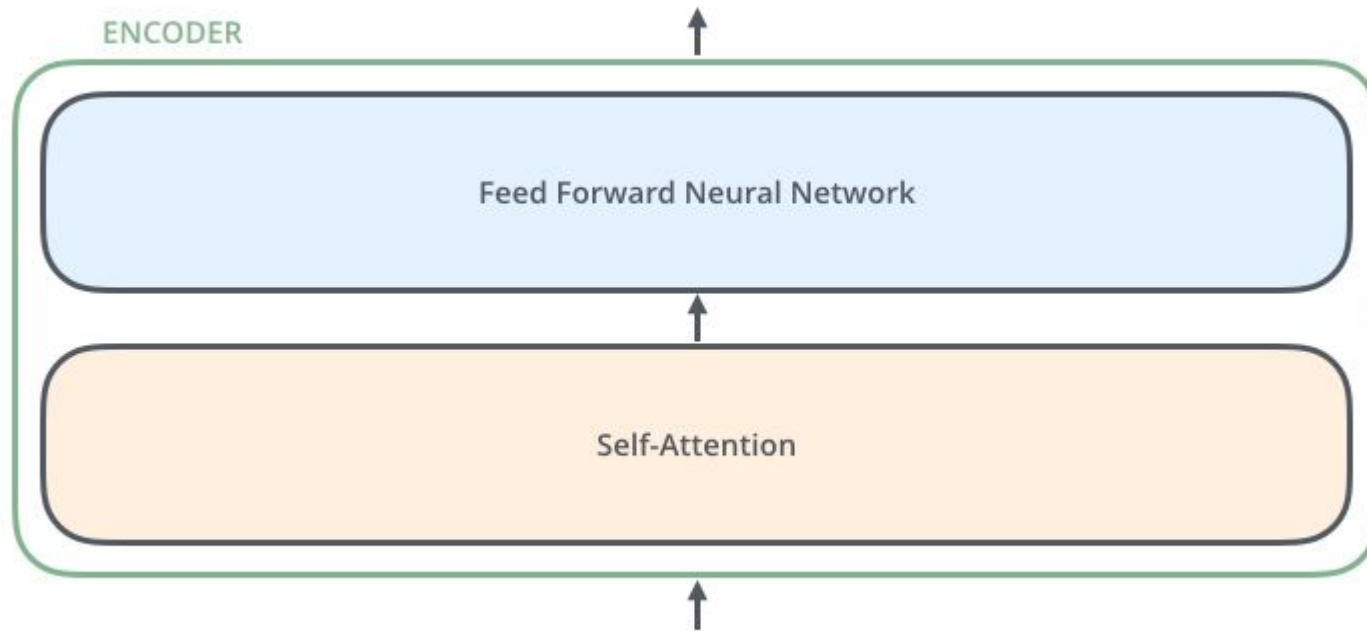


Positional Encoding

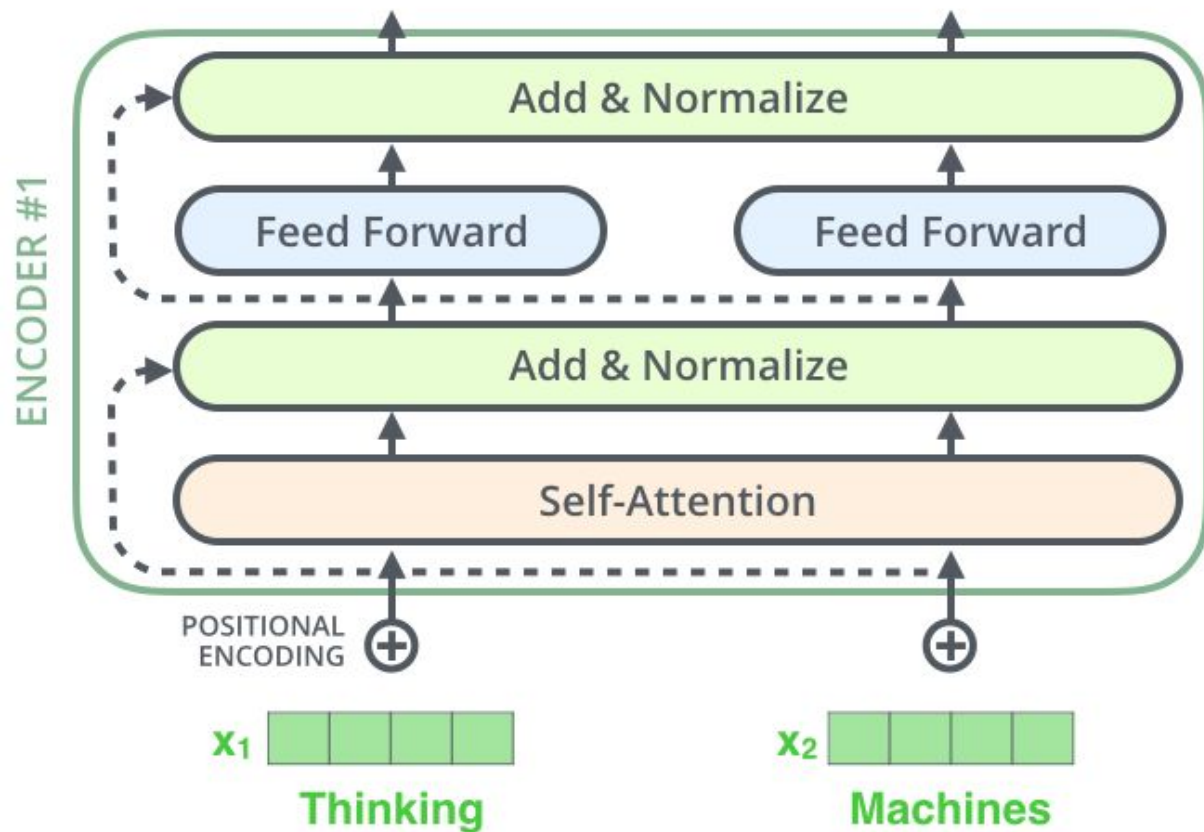


$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

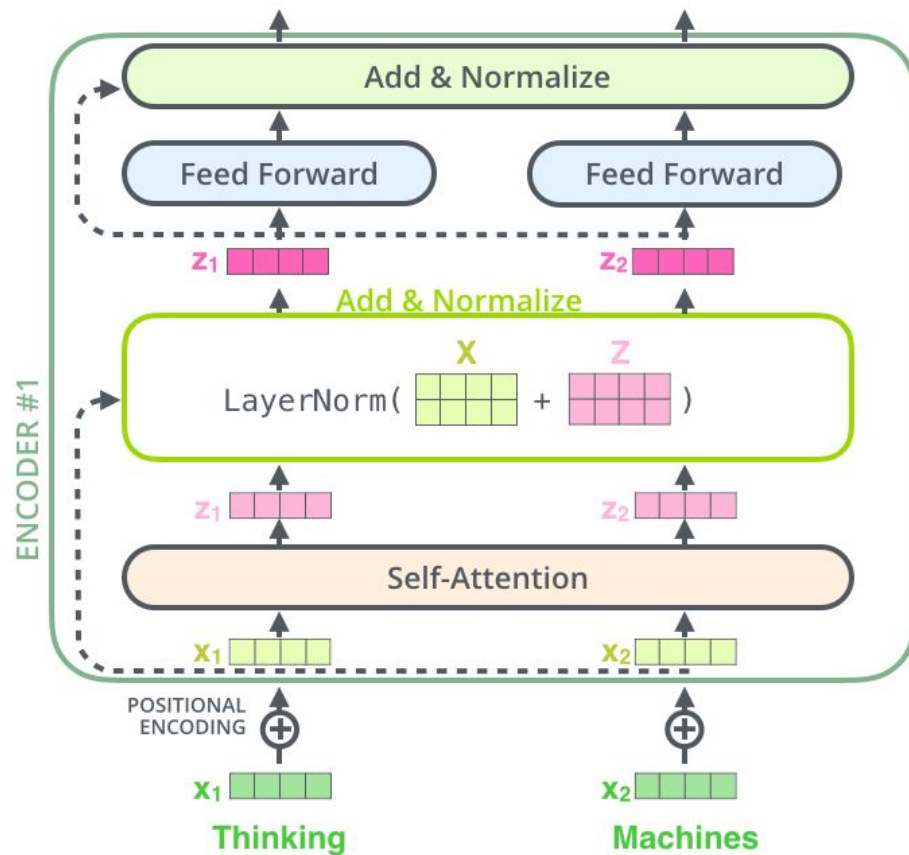
Encoder Block



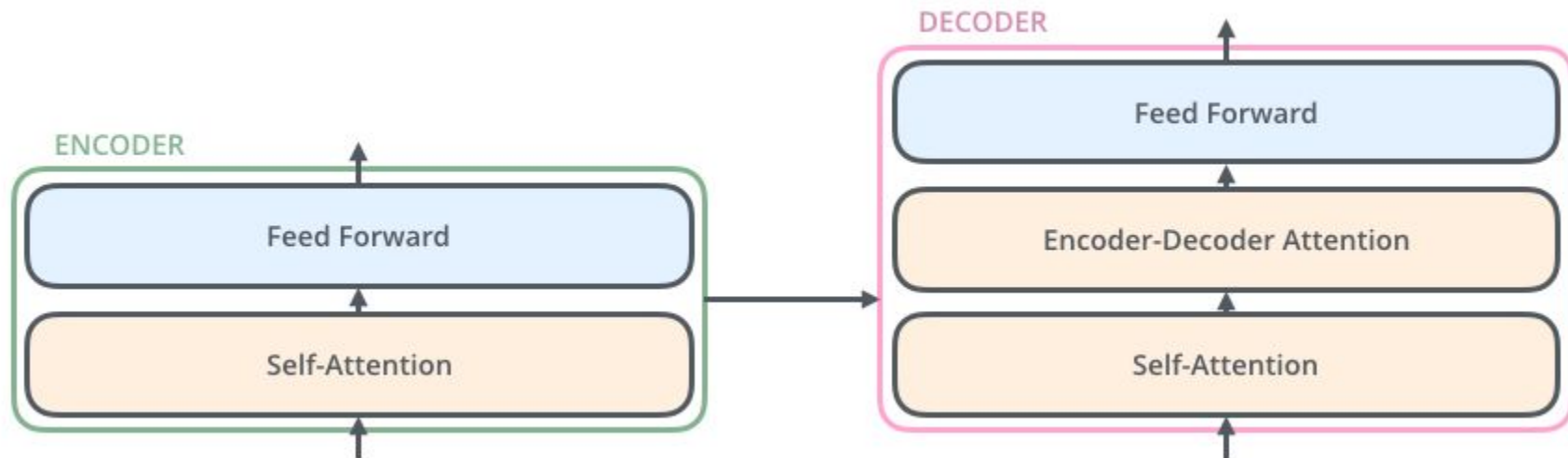
Encoder Block



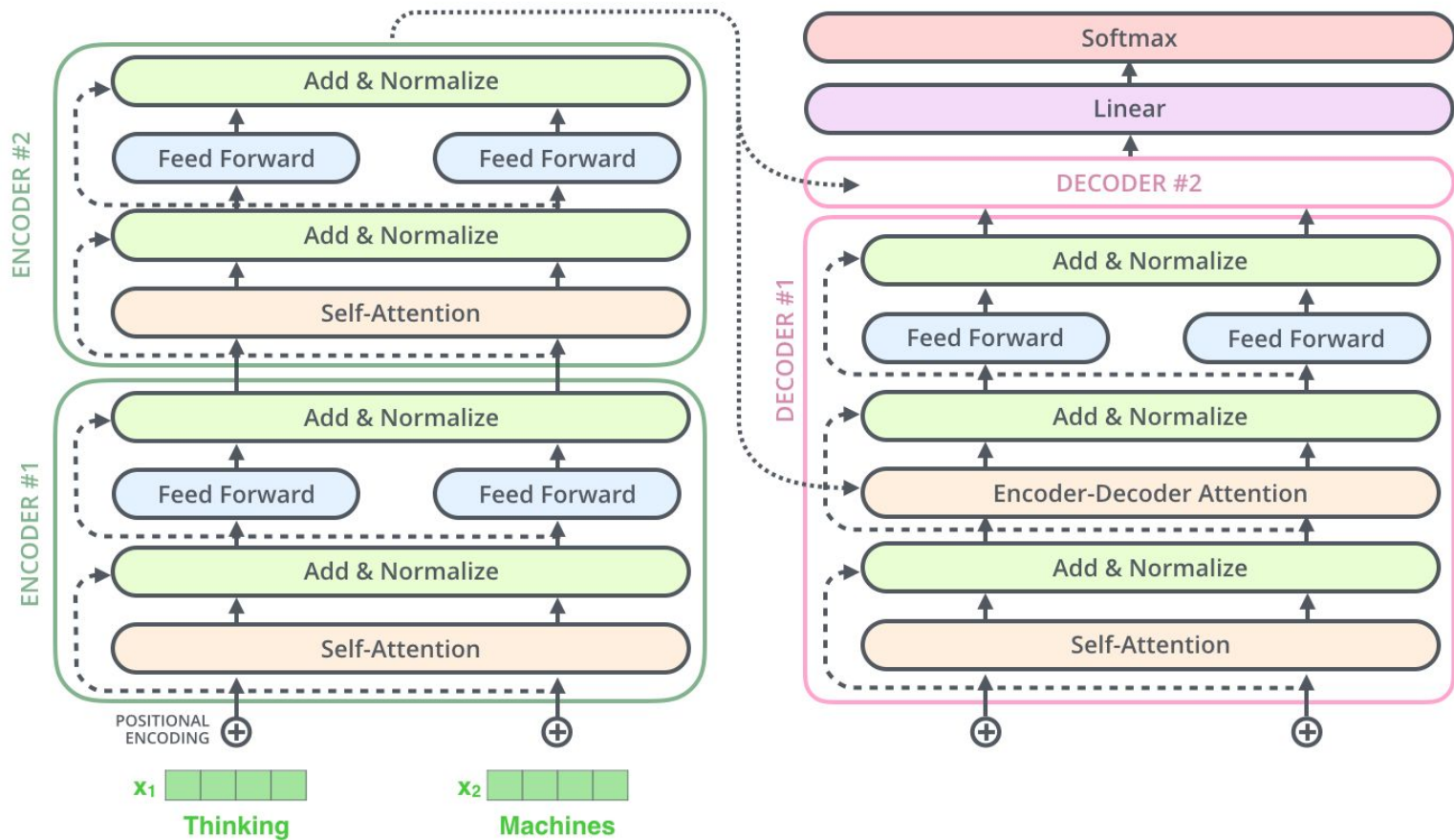
Encoder Block



Decoder Block



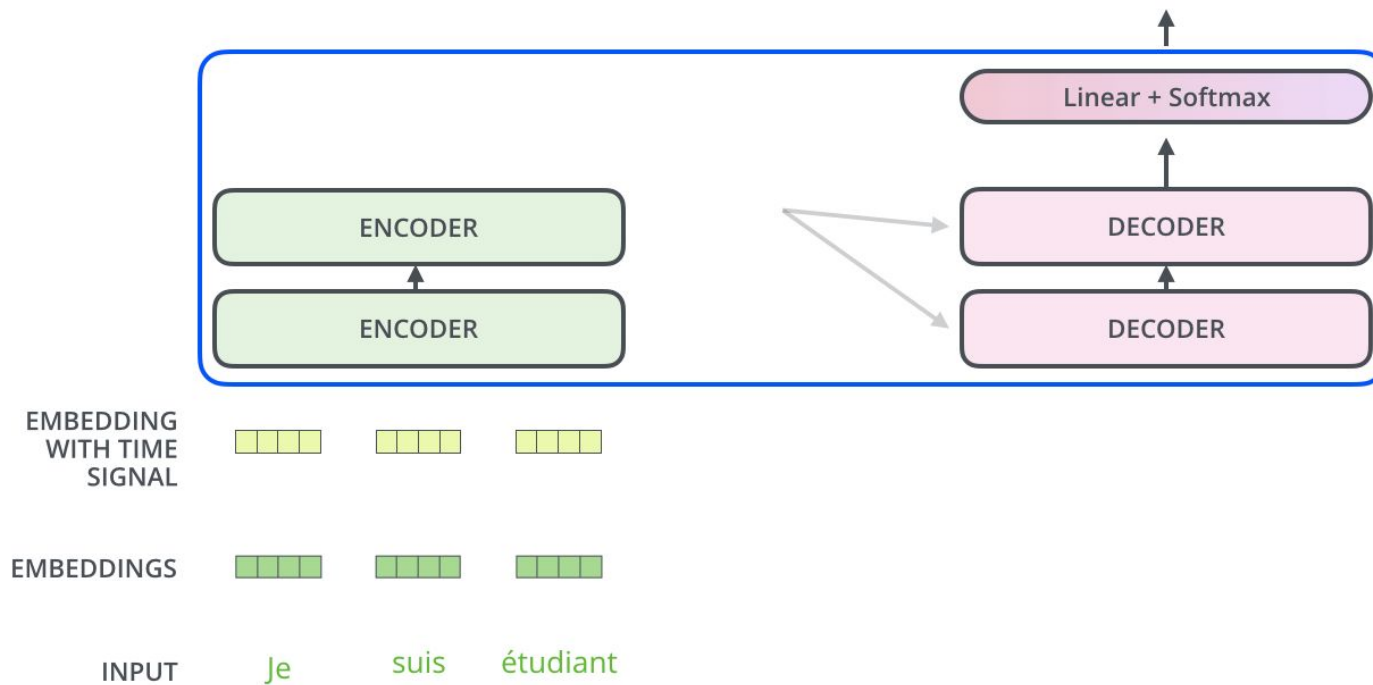
Decoder Block



Decoder

Decoding time step: 1 2 3 4 5 6

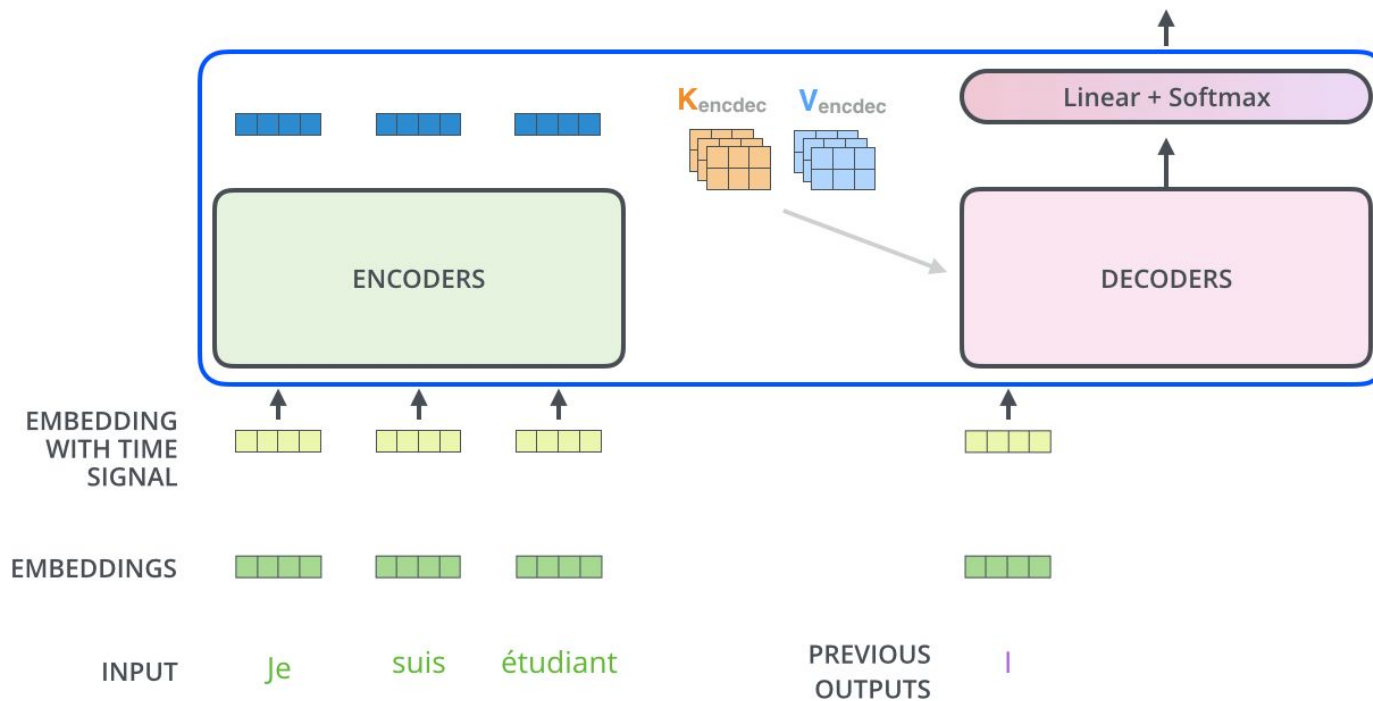
OUTPUT



Decoder

Decoding time step: 1 2 3 4 5 6

OUTPUT |



Final Linear and Softmax

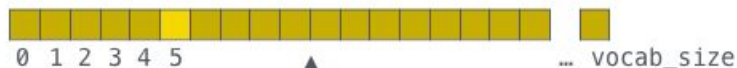
Which word in our vocabulary
is associated with this index?

am

Get the index of the cell
with the highest value
(**argmax**)

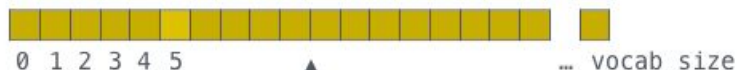
5

log_probs



Softmax

logits

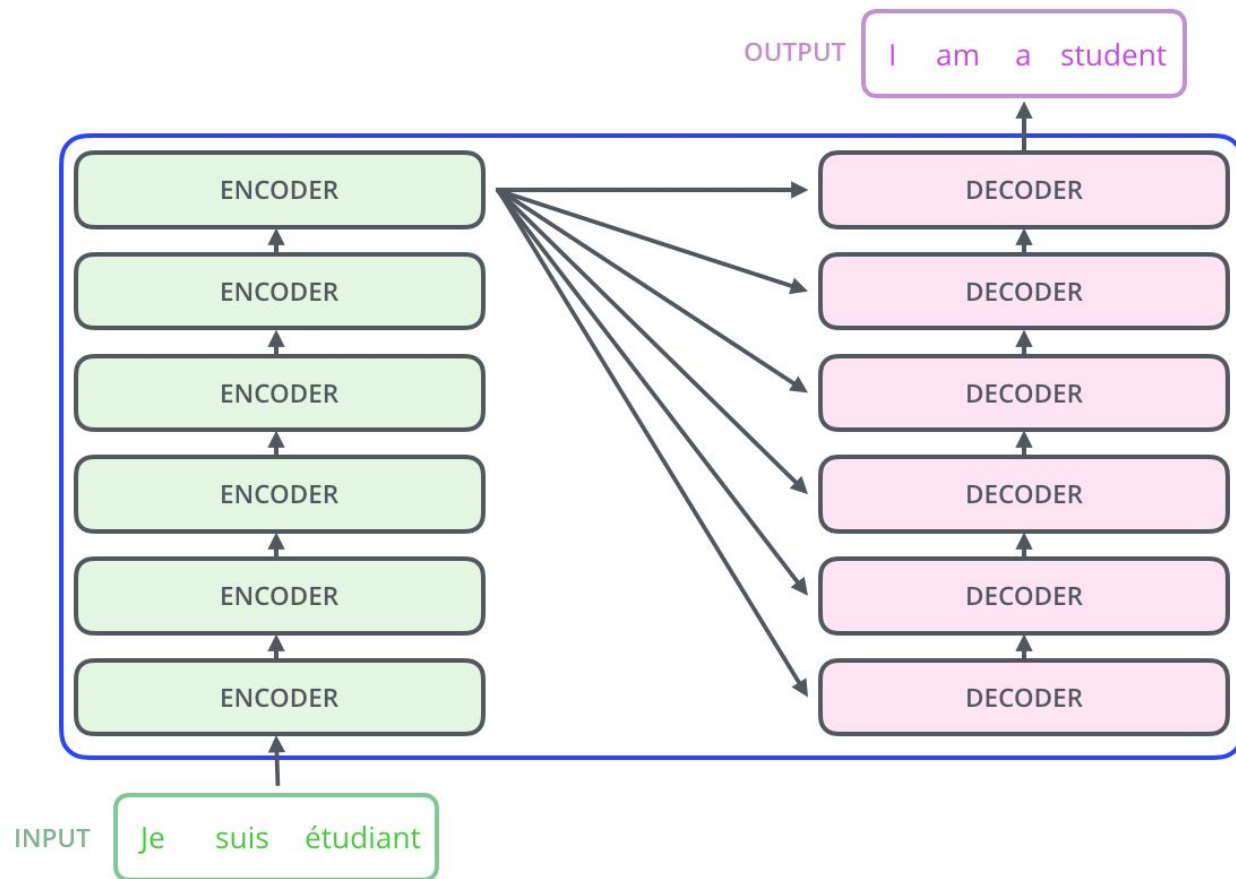


Linear

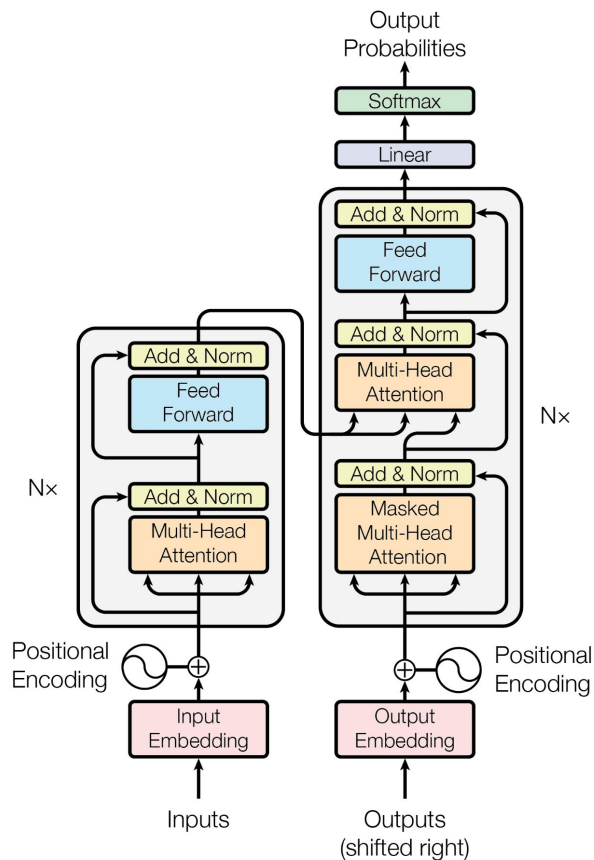
Decoder stack output



High-level Overview



Architecture Diagram from the paper



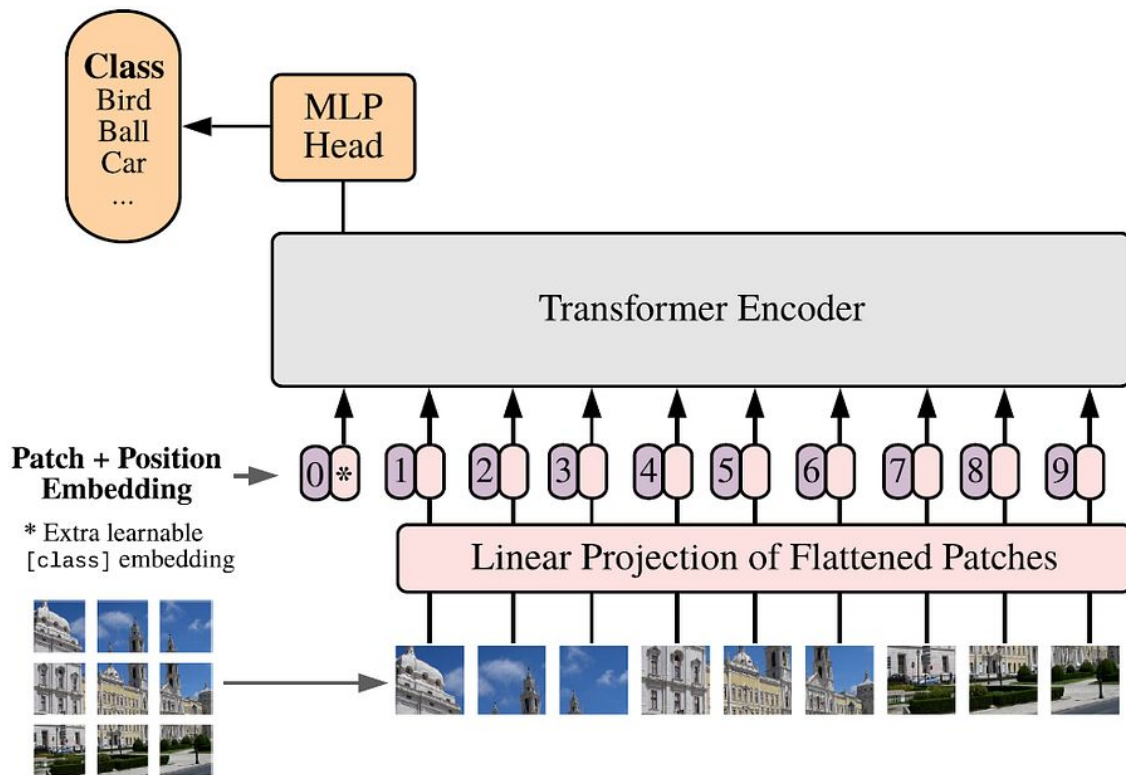
Vision Transformer – ViT

Images to tokens

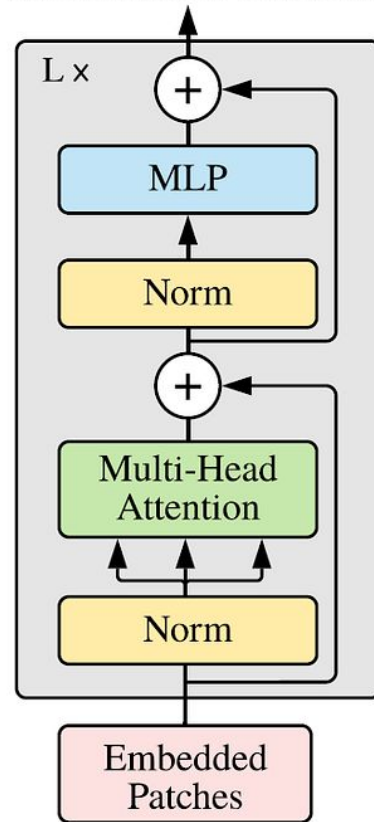


ViT Architecture

Vision Transformer (ViT)



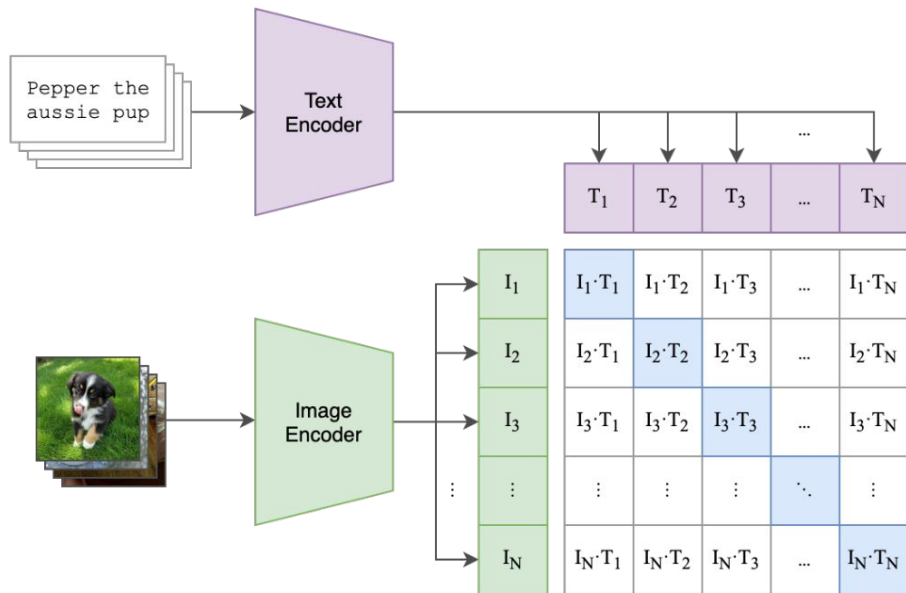
Transformer Encoder



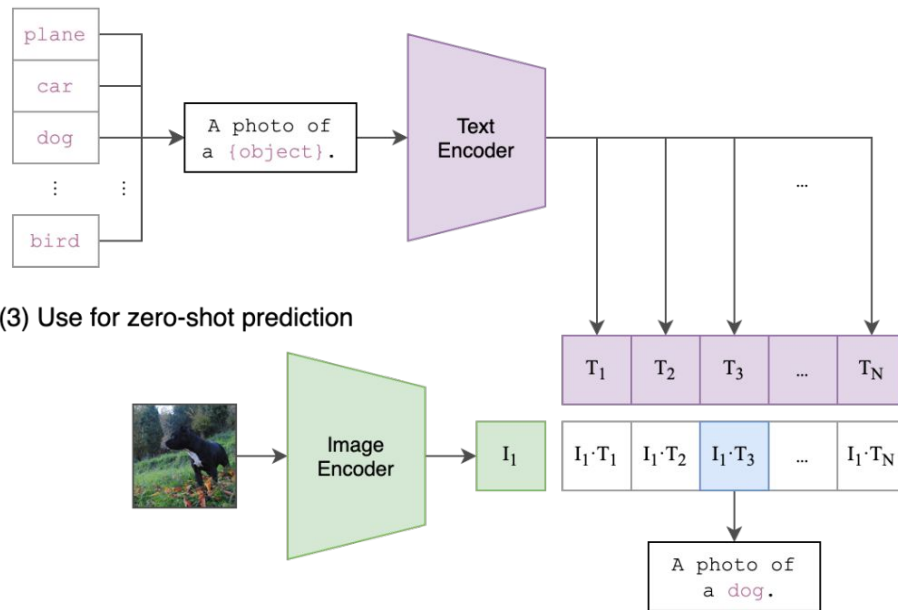
Multimodal Learning

CLIP – Contrastive Language Image Pretraining

(1) Contrastive pre-training

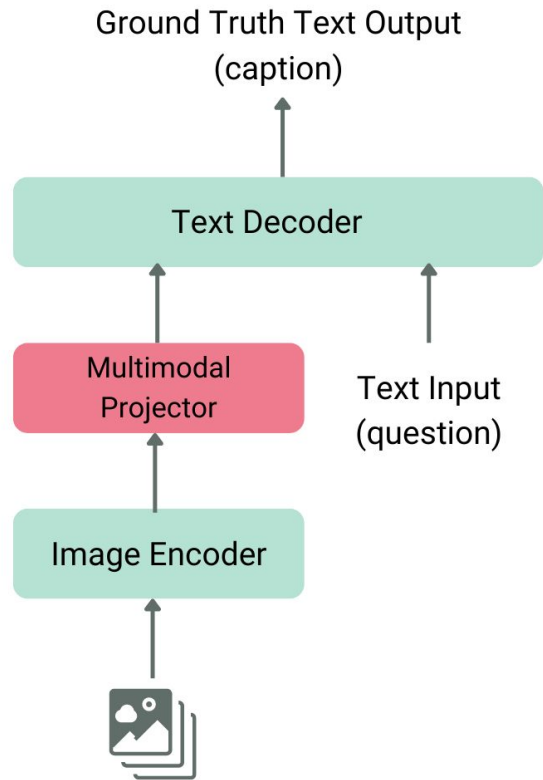


(2) Create dataset classifier from label text

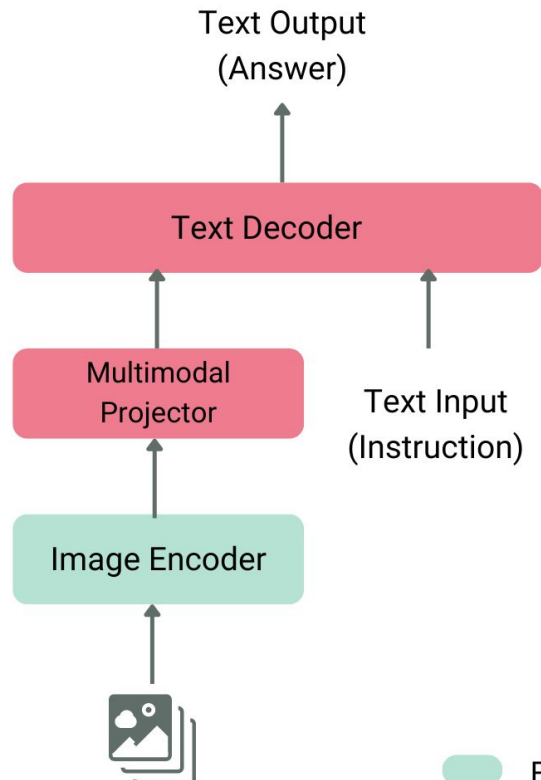


(3) Use for zero-shot prediction

Vision Language Models (VLMs)



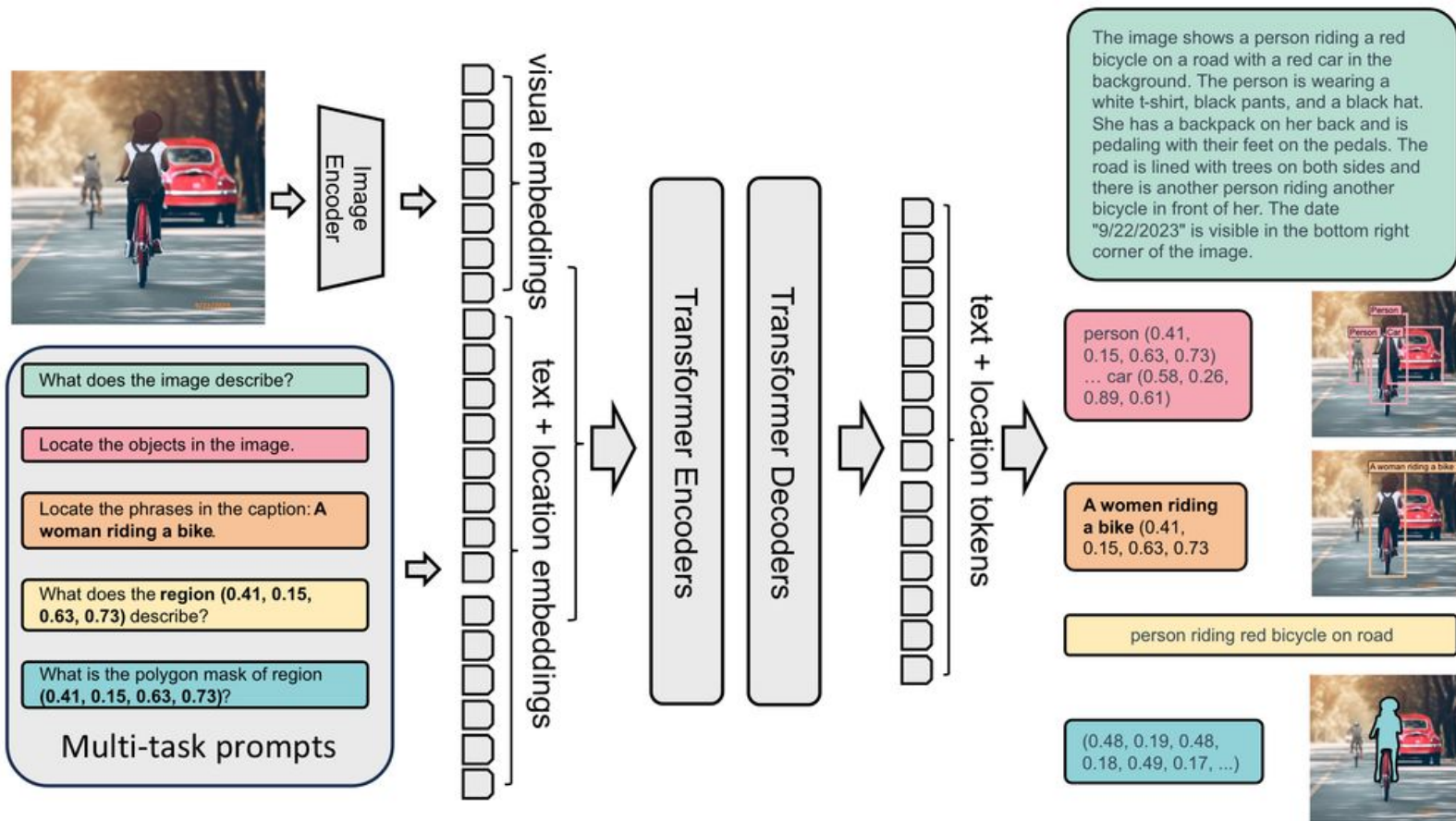
Pre-training



Fine-tuning

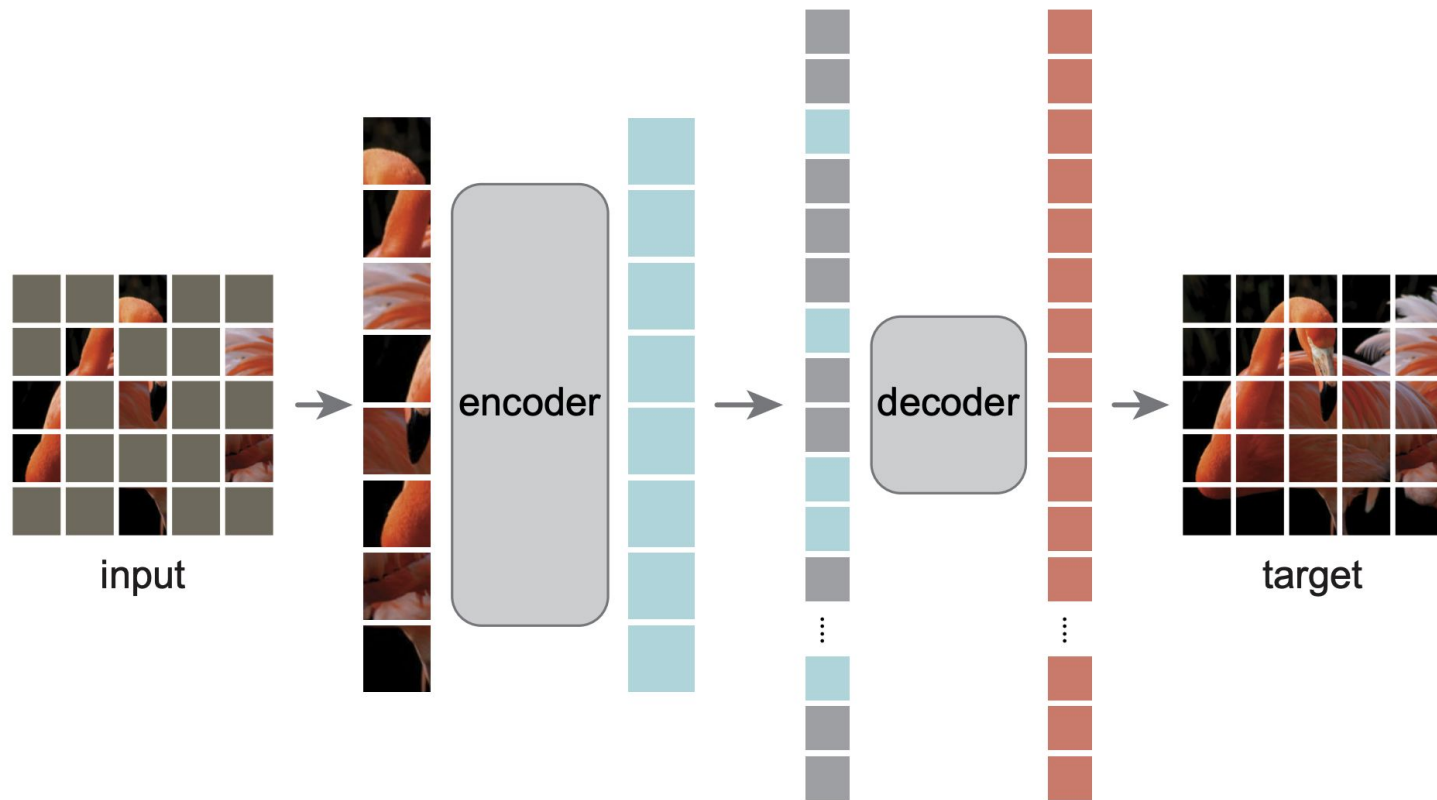
Legend:
Teal box: Frozen
Pink box: Not frozen

Capabilities of VLMs

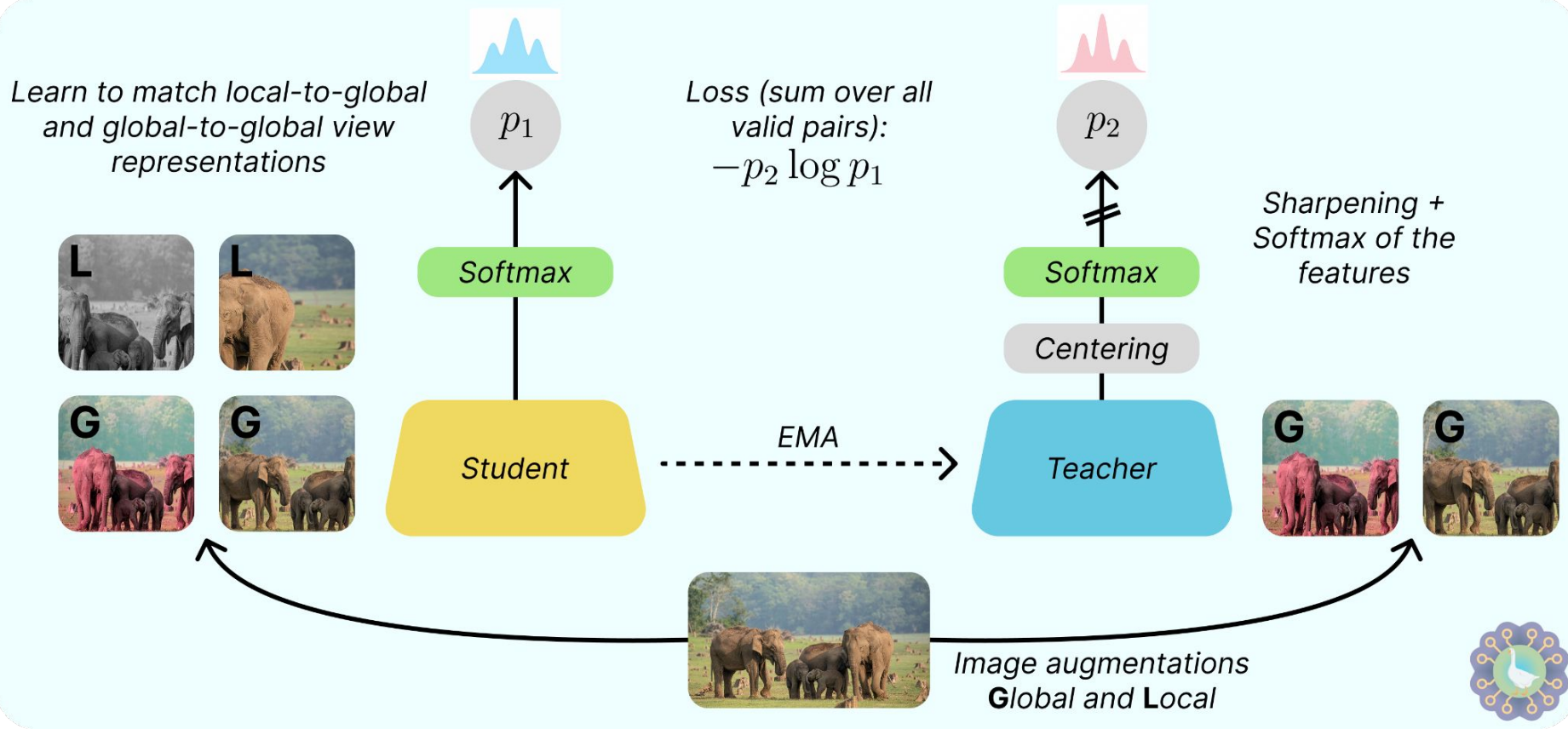


Self-Supervised Learning (SSL)

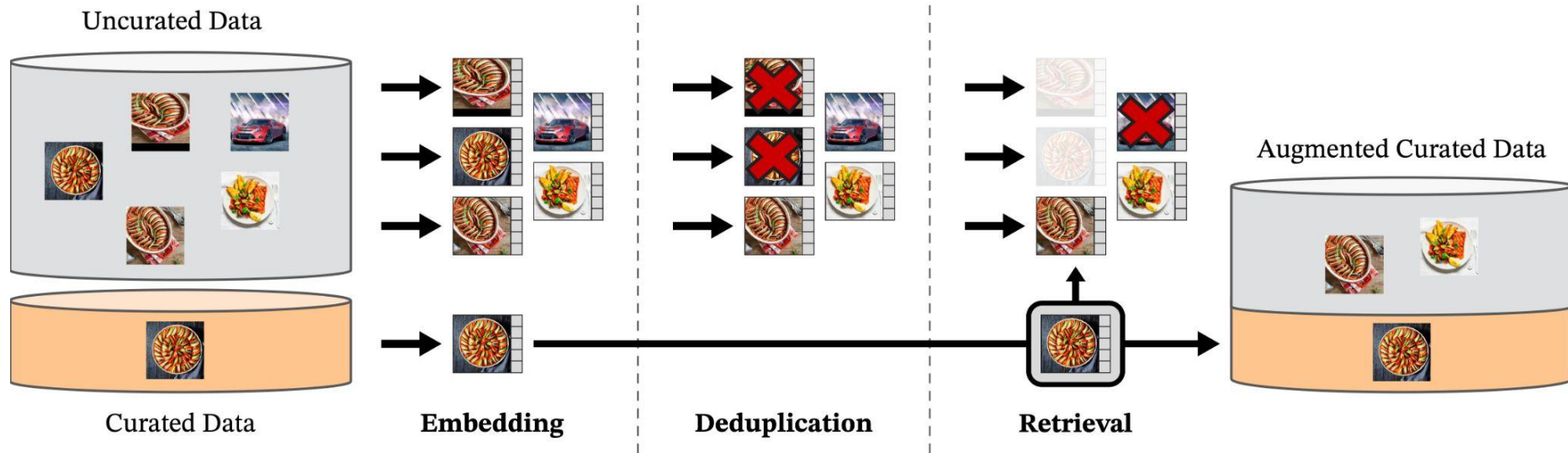
SSL Intuition



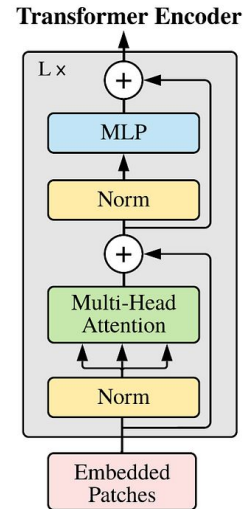
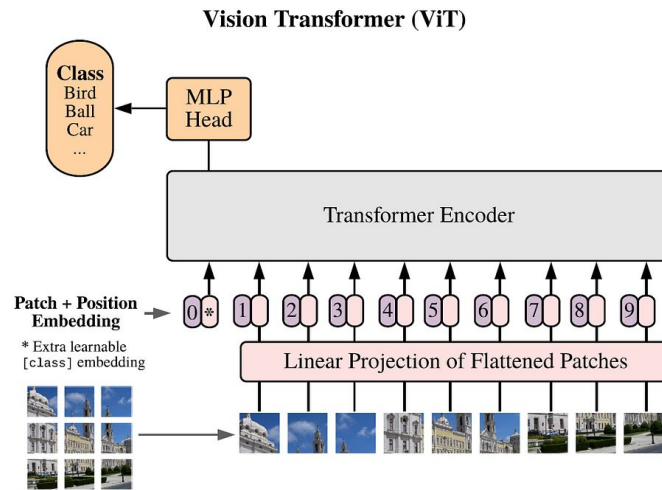
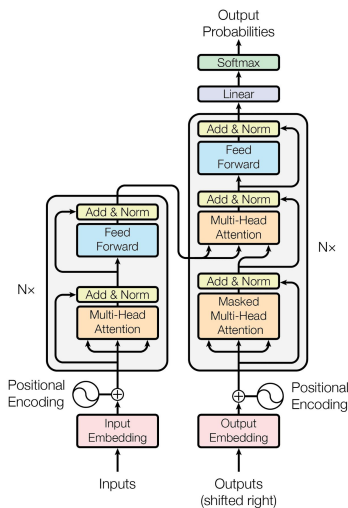
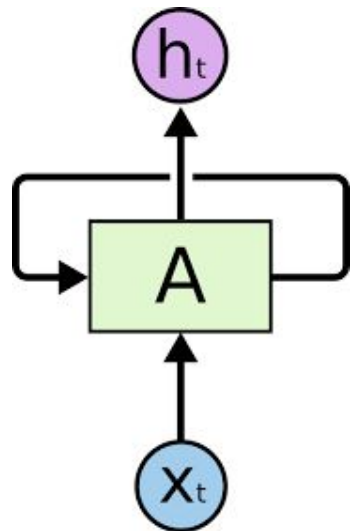
DINO



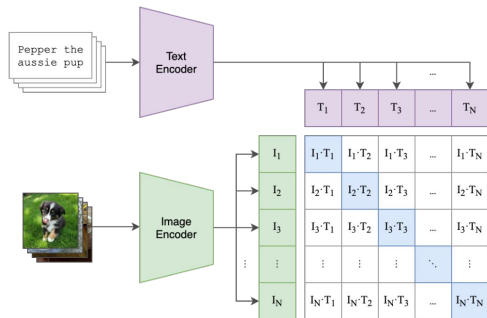
DINOv2



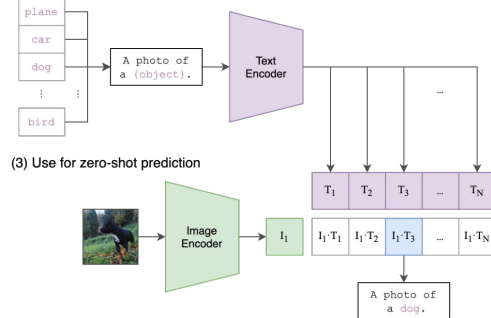
Summary



(1) Contrastive pre-training



(2) Create dataset classifier from label text



(3) Use for zero-shot prediction

